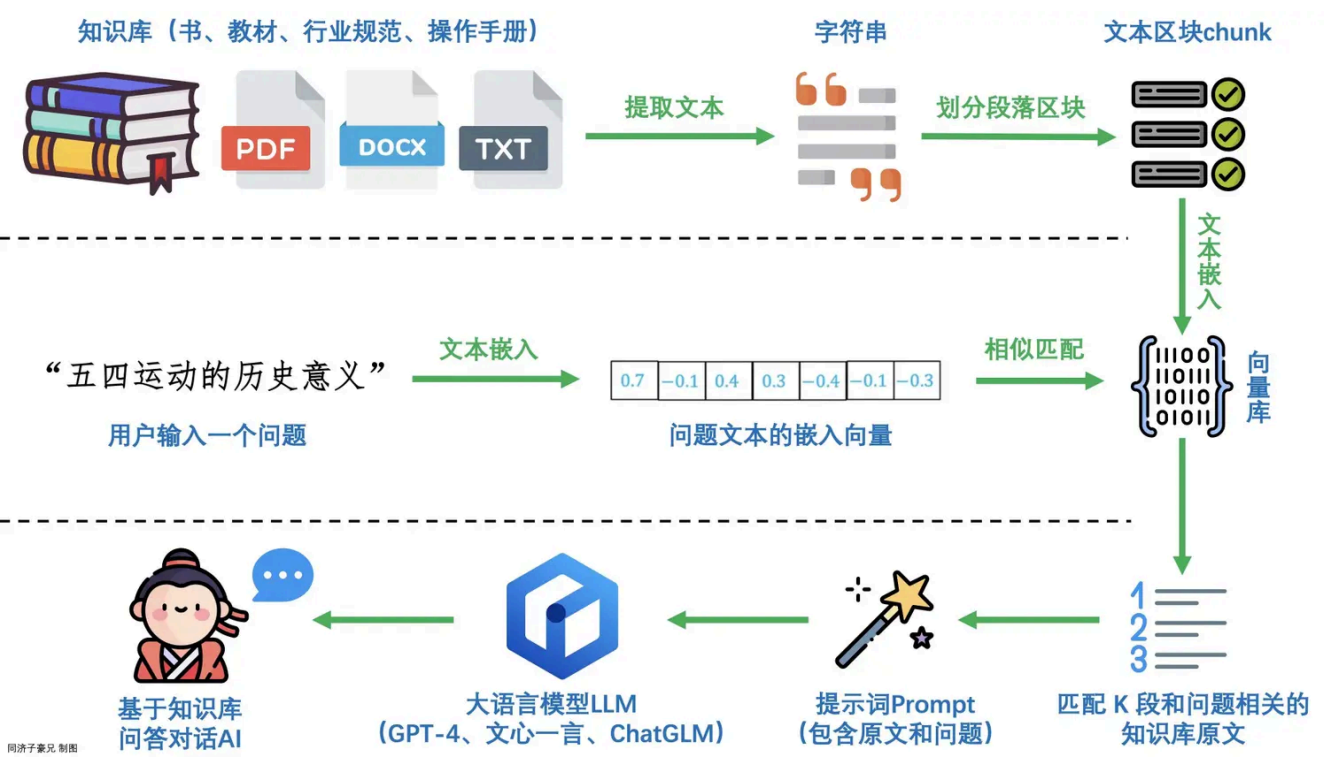


# RAG 应用开发手册（2025）

## RAG 介绍

在自然语言处理领域，大型语言模型（LLM）如 GPT-3、BERT 等已经取得了显著的进展，它们能够生成连贯、自然的文本，回答问题，并执行其他复杂的语言任务。然而，这些模型存在一些固有的局限性，如“模型幻觉问题”、“时效性问题”和“数据安全问题”。为了克服这些限制，检索增强生成（RAG）技术应运而生。

RAG 是知识库的管理，以及文本检索下的动态提示词构建过程，附加一次大模型调用。



RAG 技术结合了大型语言模型的强大生成能力和检索系统的精确性。它允许模型在生成文本时，从外部知识库中检索相关信息，从而提高生成内容的准确性、相关性和时效性。这种方法不仅增强了模型的回答能力，还减少了生成错误信息的风险。

## 大模型的局限性

大型语言模型在自然语言处理领域展示了显著的能力，但它们也存在一系列固有的缺点。首先，虽然这些模型在掌握大量信息方面非常有效，但它们的结构和参数数量使得对其进行修

改、微调或重新训练变得异常困难，且相关成本相当可观。

其次，大型语言模型的应用往往依赖于构建适当的提示（prompt）来引导模型生成所需的文本。这种方法通过将信息嵌入到提示中，从而引导模型按照特定的方向生成文本。然而，这种基于提示的方法可能使模型过于依赖先前见过的模式，而无法真正理解问题的本质。

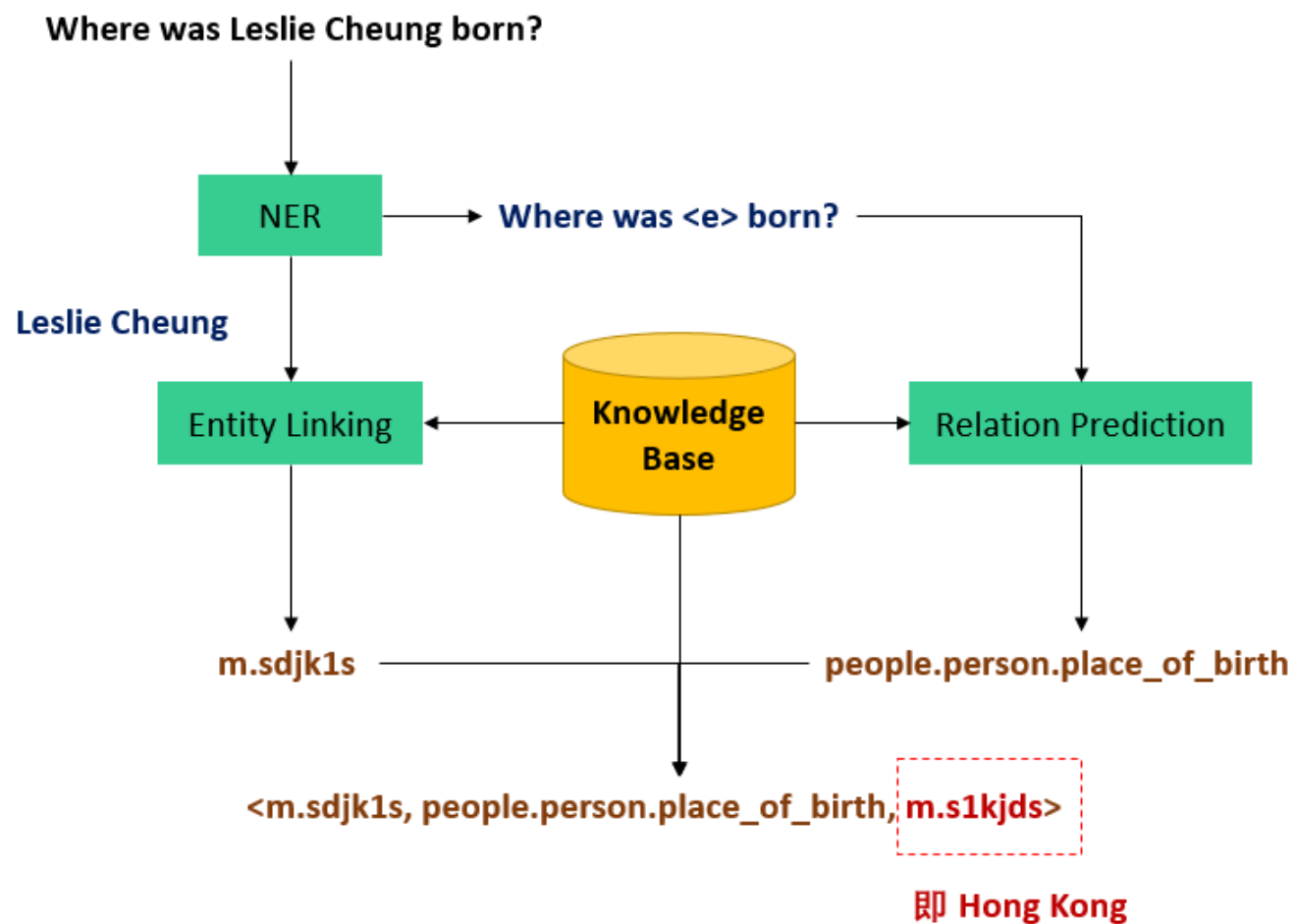
| 大模型现存问题 | 大型语言模型的局限性           |
|---------|----------------------|
| 问题 1.1  | 模型幻觉问题：生成内容可能不准确或不一致 |
| 问题 1.2  | 时效性问题：生成的内容不具有当前时效性  |
| 问题 1.3  | 数据安全问题：可能存在敏感信息泄露风险  |

在自然语言处理领域，幻觉（Hallucination）被定义为生成的内容与提供的源内容无关或不忠实，具体而言，是一种虚假的感知，但在表面上却似乎是真实的。在一般语境中，幻觉是一个心理学术语，指的是一种特定类型的感知。在自然语言处理或大型语言模型的语境下，这种感知即为一种虚假的、不符合实际的信息。

造成幻觉的原因主要可以归结为数据驱动原因、表示和解码的不完善以及参数知识偏见。首先，数据对不齐或不匹配可能导致幻觉，因为模型在训练中未能准确地理解源内容与参考内容之间的关系。

## 基于知识库问答

知识库问答（Knowledge Base Question Answering，简称 KBQA）是一种早期的对话系统方法，旨在利用结构化的知识库进行自然语言问题的回答。这种方法基于一个存储在图数据库中的知识库，通常以三元组的形式表示为 <主题，关系，对象>，其中每个三元组都附带相关的属性信息。



知识库问答早期是对话系统中的有效方法，其基于知识图谱的结构为系统提供了丰富的语义信息，使得系统能够更深入地理解用户提出的问题，并以结构化的形式回答这些问题。随着技术的不断发展，KBQA 方法也在不断演进，为对话系统的进一步提升奠定了基础。

在 KBQA 中，有两种主流方法用于处理自然语言问题：

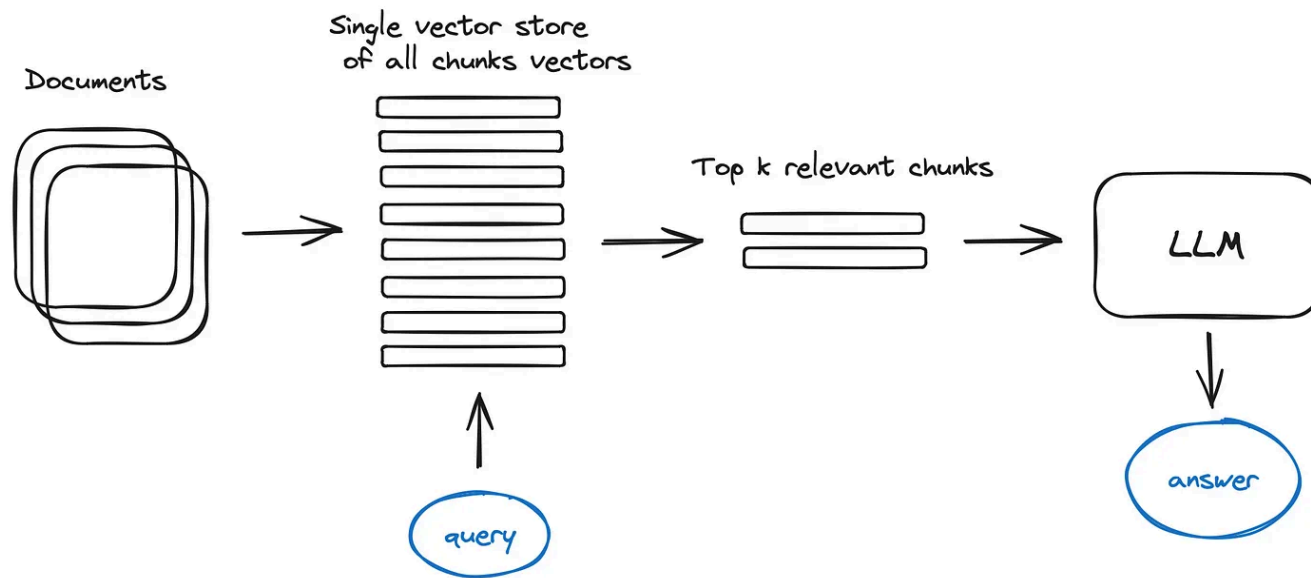
- 主题识别与实体链接：该方法从识别问题中的主题开始，将其链接到知识库中的实体（称为主题实体）。通过主题实体，系统能够在知识库中查找相关的信息并回答问题。
- 多跳查询：基于图数据库的优势，KBQA 能够进行多跳查询，即通过多个关系跨越多个实体来获取更深层次的信息。这种灵活性使得系统能够更全面地理解和回答用户的复杂问题。

## 为什么选择 RAG?

检索增强生成（RAG）技术在弥补大型语言模型（LLM）的局限性方面取得了显著进展，尤其是减少幻觉问题和提升实效性方面。在之前提到的 LLM 存在的问题中，特别是幻觉问题和时效

性问题，RAG 技术通过引入外部知识库的检索机制，成功提升了生成内容的准确性、相关性和时效性。

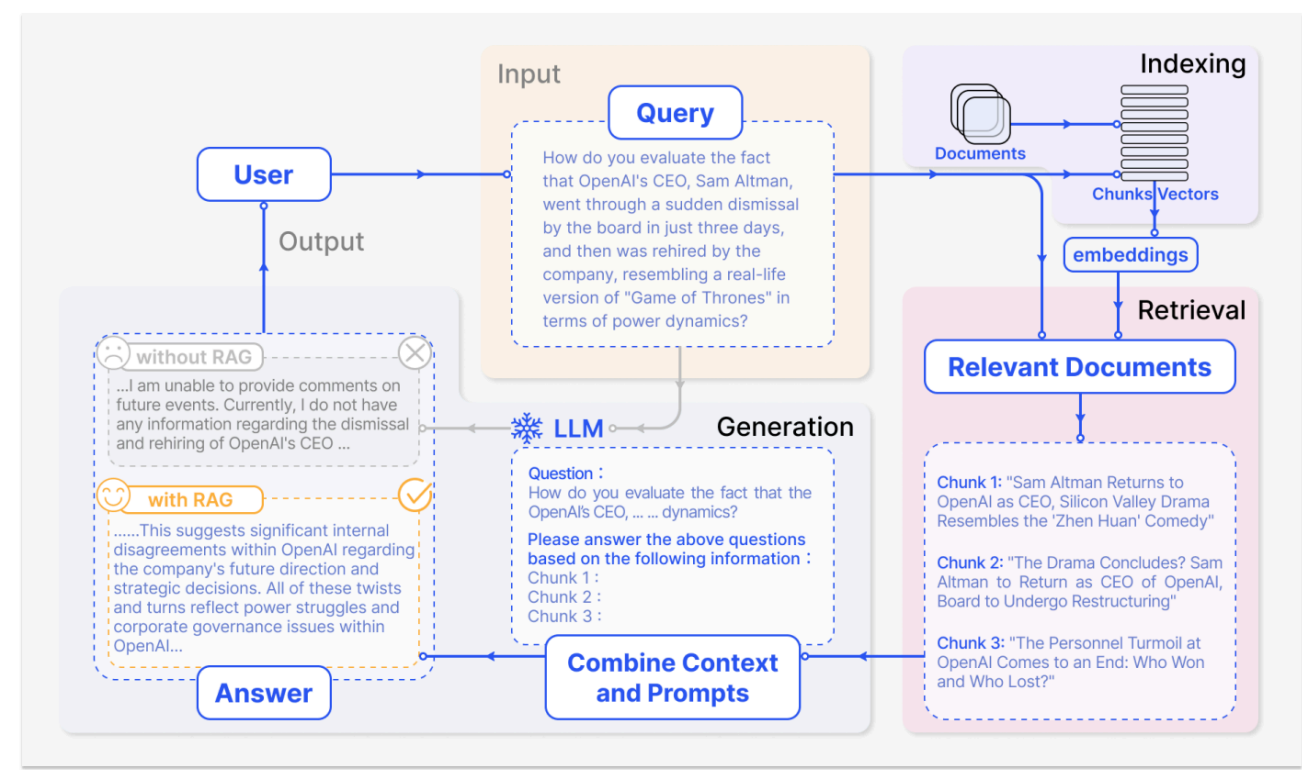
## Basic index retrieval



- RAG 技术通过检索外部知识库，避免了幻觉问题的困扰。相较于单纯依赖大型语言模型对海量文本数据的学习，RAG 允许模型在生成文本时从事实丰富的外部知识库中检索相关信息。
- RAG 技术的时效性优势使其在处理实效性较强的问题时更为可靠。通过与外部知识库的连接，RAG 确保了模型可以获取最新的信息，及时适应当前的事件和知识。
- 与传统的知识库问答（KBQA）相比，RAG 技术在知识检索方面更加灵活，不仅能够从结构化的知识库中检索信息，还能够应对非结构化的自然语言文本。

RAG 被构建为一个应用于大型语言模型的框架，其目标是通过结合大模型的生成能力和外部知识库的检索机制，提升自然语言处理任务的效果。RAG 并非旨在取代已有的知识库问答（KBQA）系统，而是作为一种补充，利用检索机制强调实时性和准确性，从而弥补大型语言模型固有的局限性。

| RAG 优点 | 描述                     |
|--------|------------------------|
| 优点 1.1 | 提高准确性和相关性              |
| 优点 1.2 | 改善时效性，使模型适应当前事件和知识     |
| 优点 1.3 | 降低生成错误风险，依赖检索系统提供的准确信息 |



RAG 框架的最终输出被设计为一种协同工作模式，将检索到的知识融合到大型语言模型的生成过程中。在应对任务特定问题时，RAG 会生成一段标准化的句子，引导大模型进行回答。下面是 RAG 输出到大型语言模型的典型模板：

Shell

1 你是一个{task}方面的专家，请结合给定的资料，并回答最终的问题。请如实回答，如果问题在资料中找不到答案，请回答不知道。

2

3 问题：{question}

4

5 资料：

6 • {information1} -》 chunk文本

7 • {information2}

8 • {information3}

其中，{task}代表任务的领域或主题，{question}是最终要回答的问题，而{information1}、{information2}等则是提供给模型的外部知识库中的具体信息。

## RAG 和 SFT 对比

在更新大型语言模型的知识方面，微调模型和使用 RAG 这两种方法有着各自的优缺点。微调模型优势在于能够通过有监督学习的方式，通过对任务相关数据的反复迭代调整，使得模型更好地适应特定领域的知识和要求。RAG 能够从外部知识库中检索最新、准确的信息，从而提高了答案的质量和时效性。其优势在于可以利用最新的外部信息，从而更好地适应当前事件和知识。

|    | 微调模型（避免微调）                        | RAG（优先考虑）                                   |
|----|-----------------------------------|---------------------------------------------|
| 优点 | 针对特定任务调整预训练模型。优点是可针对特定任务优化；       | 结合检索系统和生成模型。优点是能利用最新信息，提高答案质量，具有更好的可解释性和适应性 |
| 缺点 | 但缺点是更新成本 <sup>高</sup> ，对新信息适应性较差； | 是可能面临检索质量问题和额外计算资源需求；                       |

| 特性    | RAG 技术                | SFT 模型微调 （特定任务的时候）   |
|-------|-----------------------|----------------------|
| 知识更新  | 实时更新检索库，适合动态数据，无需频繁重训 | 存储静态信息，更新知识需要重新训练    |
| 外部知识  | 高效利用外部资源，适合各类数据库      | 可对齐外部知识，但对动态数据源不够灵活  |
| 数据处理  | 数据处理需求低               | 需构建高质量数据集，数据限制可能影响性能 |
| 模型定制化 | 专注于信息检索和整合，定制化程度低     | 可定制行为，风格及领域知识        |
| 可解释性  | 答案可追溯，解释性高            | 解释性相对低               |
| 计算资源  | 需要支持检索的计算资源，维护外部数据源   | 需要训练数据集和微调资源         |
| 延迟要求  | 数据检索可能增加延迟            | 微调后的模型反应更快           |
| 减少幻觉  | 基于实际数据，幻觉减少           | 通过特定域训练可减少幻觉，但仍然有限   |
| 道德和隐私 | 处理外部文本数据时需要考虑隐私和道德问题  | 训练数据的敏感内容可能引发隐私问题    |

## RAG 实现流程

如果使用 RAG，主要包括信息检索和大型语言模型调用两个关键过程。信息检索通过连接外部知识库，获取与问题相关的信息；而大型语言模型调用则用于将这些信息整合到自然语言生成的过程中，以生成最终的回答。

| RAG 流程        | 描述                    |
|---------------|-----------------------|
| 步骤 1（可选）：问题理解 | 准确把握用户的意图             |
| 步骤 2：知识检索     | 从知识库中相关的知识检索          |
| 步骤 3：答案生成     | 将检索结果与问题汇总在提示词中，调用大模型 |

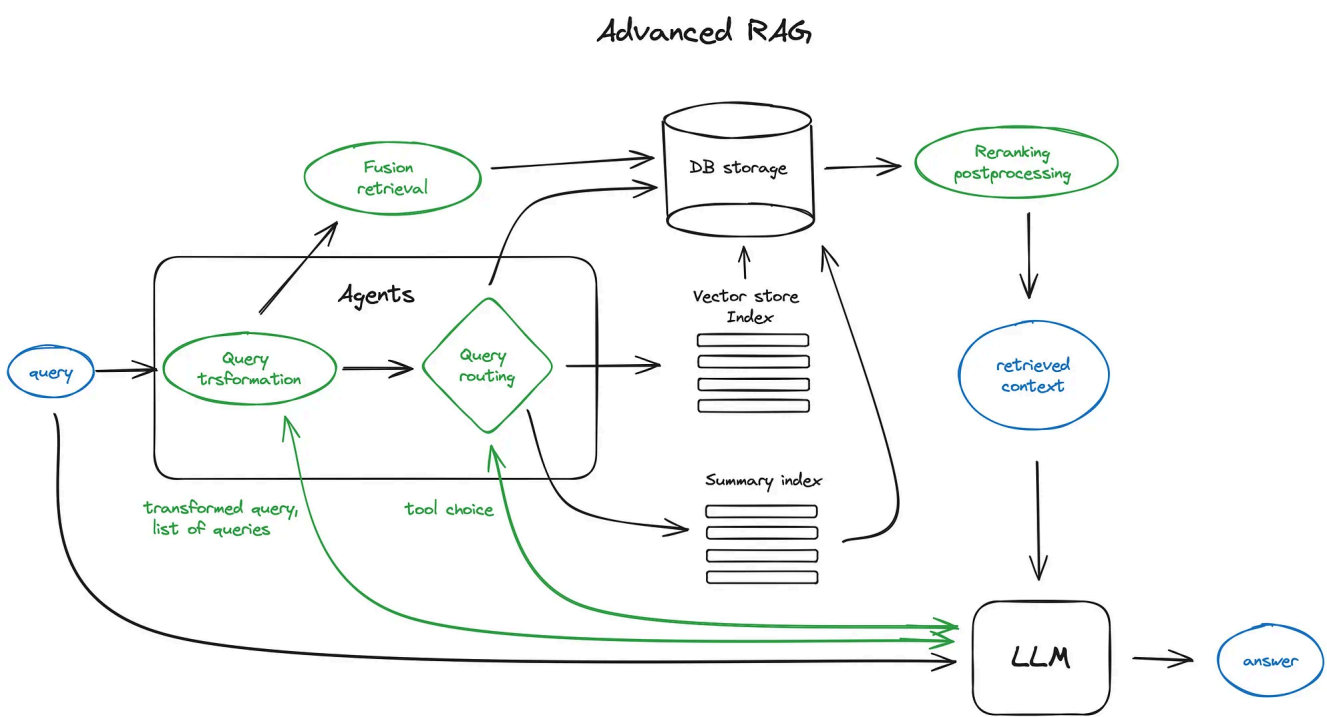
RAG 每个步骤都面临一些挑战，这些挑战使得 RAG 的实现变得复杂而困难。在问题理解阶段，系统需要准确把握用户的意图。**用户提问往往是短文本，而知识库中的信息可能是长文本。**将用户提问与知识库中的知识建立有效的关联是一个难点，特别是考虑到用户提问可能模糊，用词不规范，难以直接找到相关的知识。

- 知识检索是 RAG 流程中的关键步骤，但也是面临挑战的步骤之一。**用户提问可能以多种方式表达，而知识库的信息来源可能是多样的，包括 PDF、PPT、Neo4j 等格式。**
- 此外用户的意图可能非常灵活，可能是提问，也可能需要进行闲聊。**在这个阶段，需要确保生成的答案与用户的意图一致，同时保持自然、连贯的文本。此外，大型模型的输出可能

存在幻觉问题，即生成的内容可能与问题不相关，增加了生成准确回答的难度。

在论文综述「[Retrieval-Augmented Generation for Large Language Models: A Survey](#)」中，作者将 RAG 技术按照复杂度继续划分为 Naive RAG，Advanced RAG、Modular RAG。

| 技术类型         | 描述                                                                                                                                                                                                 |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Naive RAG    | Naive RAG 是 RAG 技术的最基本形式，也被称为经典 RAG。包括索引、检索、生成三个基本步骤。索引阶段将文档库分割成短的 Chunk，并构建向量索引。检索阶段根据问题和 Chunks 的相似度检索相关文档片段。生成阶段以检索到的上下文为条件，生成问题的回答。                                                            |
| Advanced RAG | Advanced RAG 在 Naive RAG 的基础上进行优化和增强。包含额外处理步骤，分别在数据索引、检索前和检索后进行。包括更精细的数据清洗、设计文档结构和添加元数据，以提升文本一致性、准确性和检索效率。在检索前使用问题的重写、路由和扩充等方式对齐问题和文档块之间的语义差异。在检索后通过重排序避免“Lost in the Middle”现象，或通过上下文筛选与压缩缩短窗口长度。 |
| Modular RAG  | Modular RAG 引入更多具体功能模块，例如查询搜索引擎、融合多个回答等。技术上融合了检索与微调、强化学习等。流程上对 RAG 模块进行设计和编排，出现多种不同 RAG 模式。提供更大灵活性，系统可以根据应用需求选择合适的功能模块组合。模块化 RAG 的引入使得系统更自由、灵活，适应不同场景和需求。                                          |



在 RAG 技术流程中，涉及多个关键模块，每个模块承担着特定的任务，协同工作以实现准确的知识检索和生成自然语言回答。

| 技术模块   | 描述                                                                            |
|--------|-------------------------------------------------------------------------------|
| 意图理解   | 意图理解模块负责准确把握用户提出的问题，确定用户的意图和主题。处理用户提问的模糊性和不规范性，为后续流程提供清晰的任务目标。                |
| 文档解析   | 文档解析模块用于处理来自不同来源的文档，包括 PDF、PPT、Neo4j 等格式。该模块负责将文档内容转化为可处理的结构化形式，为知识检索提供合适的输入。 |
| 文档索引   | 文档索引模块将解析后的文档分割成短的 Chunk，并构建向量索引。或通过全文索引进行文本检索，使得系统能够更快速地找到与用户问题相关的文档片段。      |
| 向量嵌入   | 向量嵌入模块负责将文档索引中的内容映射为向量表示，以便后续的相似度计算。这有助于模型更好地理解文档之间的关系，提高知识检索的准确性。            |
| 知识检索   | 知识检索模块根据用户提问和向量嵌入计算的相似度检索或文本检索打分。这一步骤需要解决问题和文档之间的语义关联，确保检索的准确性。               |
| 重排序    | 重排序模块在知识检索后对文档库进行重排序，以避免“Lost in the Middle”现象，确保最相关的文档片段在前面。                 |
| 大模型回答  | 大模型回答模块利用大型语言模型生成最终的回答。该模块结合检索到的上下文，以生成连贯、准确的文本回答。                            |
| 其他功能模块 | 可根据具体应用需求引入其他功能模块，如查询搜索引擎、融合多个回答等。模块化设计使得系统更加灵活，能够根据不同场景选择合适的功能模块组合。          |

## RAG 检索

### 文本检索流程

文本检索是一个多步骤的过程，其核心是构建倒排索引以实现高效的文本检索：

- 步骤 1（文本预处理）：在文本预处理阶段，对原始文本进行清理和规范化，包括去除停用词、标点符号等噪声，并将文本统一转为小写。接着，采用词干化或词形还原等技术，将单词转换为基本形式，以减少词汇的多样性，为后续建立索引做准备。
- 步骤 2（文本索引）：构建倒排索引是文本检索的关键步骤。通过对文档集合进行分词，得到每个文档的词项列表，并为每个词项构建倒排列表，记录包含该词项的文档及其位置信息。这种结构使得在查询时能够快速找到包含查询词的文档，为后续的文本检索奠定了基础。
- 步骤 3（文本检索）：接下来是查询处理阶段，用户查询经过预处理后，与建立的倒排索引进行匹配。计算查询中每个词项的权重，并利用检索算法（如 TFIDF 或 BM25）对文档进行

排序，将相关性较高的文档排在前面。

在实际应用中，倒排索引的构建和维护需要考虑性能问题，采用一些优化技术来提高检索效率，如压缩倒排索引、分布式索引等（ElasticSearch 文档数据库）。这些步骤共同构成了一个有序而逻辑完整的文本检索流程。

|      | 全文检索（搜索引擎）                         | 语义检索（将文本通过 BERT 编码之后计算相似度） |
|------|------------------------------------|----------------------------|
| 定义   | 通过关键词或短语匹配文本数据的过程                  | 强调理解查询与文本之间的深层语义关系         |
| 方法   | 基于关键词匹配，使用 TFIDF、BM25 等权重计算        | 使用 NLP 技术，如词嵌入、预训练的语言模型    |
| 特点   | 强调字面意义，关注表面文本的匹配                   | 关注词语之间的关联、语境和含义            |
| 应用场景 | 大规模文本数据的快速匹配                       | 对语义理解要求较高的场景               |
| 优势   | 处理速度较快，适用于大规模文本数据                  | 能够处理一词多义、近义词等语义上的复杂情况      |
| 联系   | 结合使用，先使用文本检索筛选出候选文档，然后在这些文档上应用语义检索 | 可以利用语义模型提取关键词的上下文信息，提升检索效果 |

在一些场景中，文本检索和语义检索可以结合使用，以充分利用它们各自的优势。例如，可以先使用文本检索筛选出候选文档，然后在这些文档上应用语义检索来进一步提高检索的准确性。当然具体使用哪种检索方法，需要具体分析，在 RAG 中可以结合两种方法一起进行使用。

## 全文检索

### 反向索引

反向索引（Inverted Index）是信息检索领域中一种**核心的数据结构**，它就像一本书的**词语 - 页码索引**。想象一下，你有一本书，你想快速找到所有提到某个词的页面。**传统的阅读方式是逐页翻阅，效率很低。而书后的索引会告诉你每个词出现在哪些页面，让你能迅速定位。**反向索引在数字世界中扮演的就是这个角色。

# inverted index

```
{
  "type": "book",
  "title": "linux kernel",
  "author": "alden",
  "rating": 4.0
}
```

```
{
  "type": "book",
  "title": "elasticsearch essential",
  "author": "alden",
  "rating": 5.0
}
```

| Tokens        | Documents |
|---------------|-----------|
| book          | 1, 2      |
| linux         | 1         |
| kernel        | 1         |
| alden         | 1, 2      |
| elasticsearch | 2         |
| essential     | 2         |

反向索引的核心思想是从文档到词语的映射，转变为从词语到文档的映射。反向索引的构建过程大致如下：

- **文档预处理：** 首先，系统会对原始文档进行处理，比如**分词**（把连续的文本切分成一个个独立的词语或短语），**标准化**（比如把所有字母转成小写，去除标点符号等），以及**去除停用词**（移除像 “的”、“是”、“和” 这类意义不大的常用词）。
- **构建倒排列表：** 对于每个处理后的词语，系统会记录它出现在哪些文档中，以及在这些文档中的具体位置信息（比如行号、词语在文档中的偏移量等）。这个记录就是**倒排列表**。

反向索引之所以被广泛应用于搜索引擎、数据库和各种检索系统中，是因为它提供了以下关键优势：

- **高效检索：** 当用户输入查询词时，系统可以直接通过反向索引找到包含这些词的所有文档，而无需扫描所有文档。这大大加快了检索速度。
- **支持复杂查询：** 它可以轻松支持 “与”、“或”、“非” 等布尔逻辑查询。例如，要查找同时包含 “人工智能” 和 “应用” 的文档，系统只需找到这两个词的倒排列表，然后取它们的交集即可。
- **排名优化：** 结合 TF-IDF、BM25 等算法，反向索引还能帮助评估文档与查询的相关性，从而对搜索结果进行排序，把最相关的文档展示给用户。

```
1 假如有100w个文档 -》 分词 -》 剔除停用词 -》 倒排索引
2
3 用户的提问：机器学习与深度学习的区别？
4     -》分词：机器学习 与 深度学习 的 区别？
5     -》剔除停用词：机器学习 与 深度学习 区别
6     -》倒排索引进行检索：
7         -》 机器学习： 包含机器学习的文档 10个
8         -》 深度学习： 包含机器学习的文档 20个
9         -》 区别： 包含区别文档 1000个
10
11     -》TFIDF打分 / BM25打分 -》 top10文档
12
13     -》语义编码 + 语义检索 -》 top10文档
14
15     -》多路合并 -》 top5文档 -》 重排序 bert-nsp
16
17 考察：全文检索的实现逻辑 、 elasticsearch的原理
```

TFIDF

TFIDF（Term Frequency-Inverse Document Frequency）是一种用于信息检索和文本挖掘的常用权重计算方法，旨在衡量一个词项对于一个文档集合中某个文档的重要性。该方法结合了两个方面的信息：词项在文档中的频率（TF）和在整个文档集合中的逆文档频率（IDF）。

- 词项在文档中的频率（TF）：

$$TF(t,d) = \frac{\text{词项}t\text{在文档}d\text{中出现的次数}}{\text{文档}d\text{中所有词项的总数}}$$

其中，t表示词项，d表示文档。TF表示了一个词项在文档中的相对频率，即在文档中出现的次数相对于文档总词项数的比例。

- 逆文档频率（IDF）：

$$IDF(t) = \log \left( \frac{\text{文档集合中的文档总数}}{\text{包含词项}t\text{的文档数}} \right)$$

其中， $t$  表示词项。IDF 表示了一个词项在整个文档集合中的稀有程度，如果词项在许多文档中都出现，其 IDF 值较低，反之则较高。

- **TFIDF 的计算：**

$$TFIDF(t,d,D) = TF(t,d) \times IDF(t)$$

其中， $D$  表示文档集合。TFIDF 的最终值是将词项在文档中的频率和在整个文档集合中的逆文档频率相乘，这样可以得到一个更全面的评估，既考虑了在文档中的重要性，也考虑了在整个文档集合中的稀有性。

## BM25

BM25Okapi 是 BM25 算法的一种变体，它在信息检索中用于评估文档与查询之间的相关性。BM25Okapi 的主要参数包括：

- $k_1$ ：控制词项频率对分数的影响，通常设置为 1.5。
- $b$ ：控制文档长度对分数的影响，通常设置为 0.75。
- $\epsilon$ ：用于防止逆文档频率（IDF）为负值的情况，通常设置为 0.25。

BM25Okapi 的打分过程基于以下三个因素：词项在文档中的频率（TF）、文档的长度（ $doc\_len$ ）以及逆文档频率（IDF）。

- TF（词项在文档中的频率）
- IDF（逆文档频率）
- 文档长度（ $doc\_len$ ）

文档长度对分数的影响通过  $b$  控制。文档长度越长，该项的分数越小。BM25Okapi 的打分公式综合考虑了以上三个因素，通过对每个词项的打分求和得到最终的文档与查询的相关性分数。

$$score = \sum_{q \in query} \left( IDF(q) \cdot \frac{q_{freq} \cdot (k_1 + 1)}{q_{freq} + k_1 \cdot (1 - b + b \cdot \frac{doc}{avgdl})} \right)$$

其中，`avgdl` 是文档集合中的平均文档长度。BM25Okapi 通过合理调整参数，兼顾了词项频率、文档长度和逆文档频率，使得在信息检索任务中能够更准确地评估文档与查询之间的相关性，提高检索效果。

## 语义检索

语义检索是通过词嵌入和句子嵌入等技术，将文本表示为语义丰富的向量。通过相似度计算和结果排序找到最相关的文档。用户查询经过自然语言处理处理，最终系统返回经过排序的相关文档，提供用户友好的信息展示。语义检索通过深度学习和自然语言处理技术，使得系统能够更准确地理解用户查询，提高检索的准确性和效果。

## 文本编码模型

文本编码模型对于语义检索的精度至关重要。目前，大多数语义检索系统采用预训练模型进行文本编码，其中最为常见的是基于 BERT（Bidirectional Encoder Representations from Transformers）的模型，或者使用 GPT（Generative Pre-trained Transformer）等。这些预训练模型通过在大规模语料上进行训练，能够捕捉词语和句子之间的复杂语义关系。选择合适的文本编码模型直接影响到得到的文本向量的有效性，进而影响检索的准确性和效果。

编码模型排行榜：<https://huggingface.co/spaces/mteb/leaderboard>

## 文本切分方法（全文检索不需要切分）

- 有哪些文本切分的方法？
- 你在项目中使用了哪一个文本切分方法？

文本 1: 我爱机器学习，我爱大模型。 -》 bert -》 向量 1

文本 2: 我爱机器学习，我爱大模型。我爱机器学习，我爱大模型。 -》 bert -》 向量 2

文本的长度是另一个关键因素，影响了文本编码的结果。短文本和长文本在编码成向量时可能表达不同的语义信息。即使两者包含相同的单词或有相似的语义，由于上下文的长度不同，得

到的向量也会有所不同。因此，当在语义检索中使用短文本来检索长文本时，或者反之，可能导致一定的误差。针对文本长度的差异，有些系统采用截断或填充等方式处理，以保持一致的向量表示。

| 名称                      | 分割依据              | 描述                                                             |
|-------------------------|-------------------|----------------------------------------------------------------|
| 递归式分割器<br>(段落、句子、chunk) | 一组用户定义的字符         | 递归地分割文本。递归分割文本的目的是尽量保持相关的文本段落相邻。这是开始文本分割的推荐方式。                 |
| HTML 分割器                | HTML 特定字符         | 基于 HTML 特定字符进行文本分割。特别地，它会添加有关每个文本块来源的相关信息（基于 HTML 结构）。         |
| Markdown 分割器            | Markdown 特定字符     | 基于 Markdown 特定字符进行文本分割。特别地，它会添加有关每个文本块来源的相关信息（基于 Markdown 结构）。 |
| 代码分割器                   | 代码（Python、JS）特定字符 | 基于特定于编码语言的字符进行文本分割。支持从 15 种不同的编程语言中选择。                         |
| Token 分割器               | Tokens            | 基于 Token 进行文本分割。存在一些不同的 Token 计量方法。                            |
| 字符分割器                   | 用户定义的字符           | 基于用户定义的字符进行文本分割。这是较为简单的分割方法之一。                                 |

对于绝大部分的文档（自然语言），可以推荐使用字符分割器，结合 Chunk Size 和 Overlap Size 可以得到不同的切分：

- **Chunk Size（块大小）**：表示将文本划分为较小块的大小。这是分割后每个独立文本块的长度或容量。块大小的选择取决于应用的需求和对文本结构的理解。
- **Overlap Size（重叠大小）**：指相邻两个文本块之间的重叠部分的大小。在切割文本时，通常希望保留一些上下文信息，重叠大小就是控制这种上下文保留的参数。

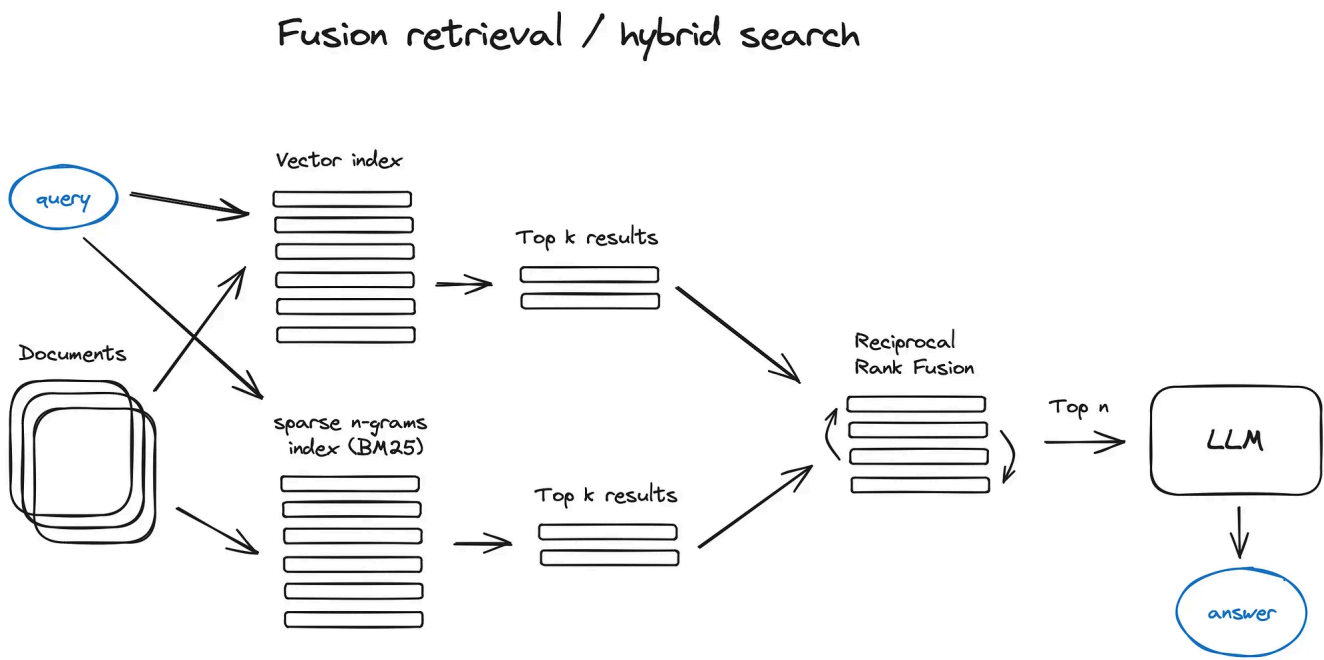
更多阅读资料：

- [https://python.langchain.com/v0.1/docs/modules/data\\_connection/document\\_transformers/](https://python.langchain.com/v0.1/docs/modules/data_connection/document_transformers/)
- <https://chunkviz.up.railway.app/>

## 多路召回

多路召回逻辑是在文本检索中常用的一种策略，其目的是通过多个召回路径（或方法）综合获取候选文档，以提高检索的全面性和准确性。单一的召回方法可能由于模型特性或数据特点而存在局限性，多路召回逻辑引入了多个召回路径，每个路径采用不同的召回方法。

- 实现方法 1：将 BM25 的检索结果 和 语义检索结果 按照排名进行加权
- 实现方法 2：按照段落、句子、页不同的角度进行语义编码进行检索，综合得到检索结果。

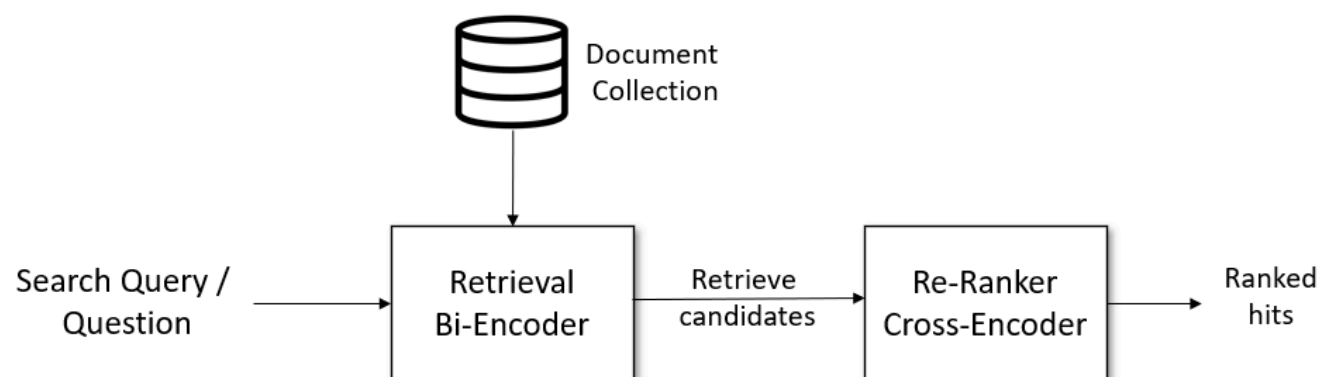


## 重排序（Rerank）

排序：按照打分的对检索结果进行排序

重排序：在排序结果的基础上重新打分，进行排序

重排序逻辑是文本检索领域中一种重要的策略，主要用于优化原有文本检索方法返回的候选文档顺序，以提高最终的检索效果。在传统的文本检索方法中，往往采用打分的逻辑，如计算 BERT 嵌入向量之间的相似度。而重排序逻辑引入了更为复杂的文本交叉方法，通过特征交叉得到更进一步的打分，从而提高排序的准确性。



- 重排序逻辑常常使用更为强大的模型，如交叉编码器（cross-encoder）模型。这类模型能够更好地理解文本之间的交叉关系，捕捉更复杂的语义信息。
- 首先通过传统的嵌入模型获取初始的 Top-k 文档，然后使用重排序逻辑对这些文档进行重新排序。这样可以在保留初步筛选文档的基础上，更精确地排列它们的顺序。

rerank 模型（2 分类模型）：通过了微调之后的模型

- query [nsp] 待选文本，匹配，标签 1
- query [nsp] 待选文本，不匹配，标签 0

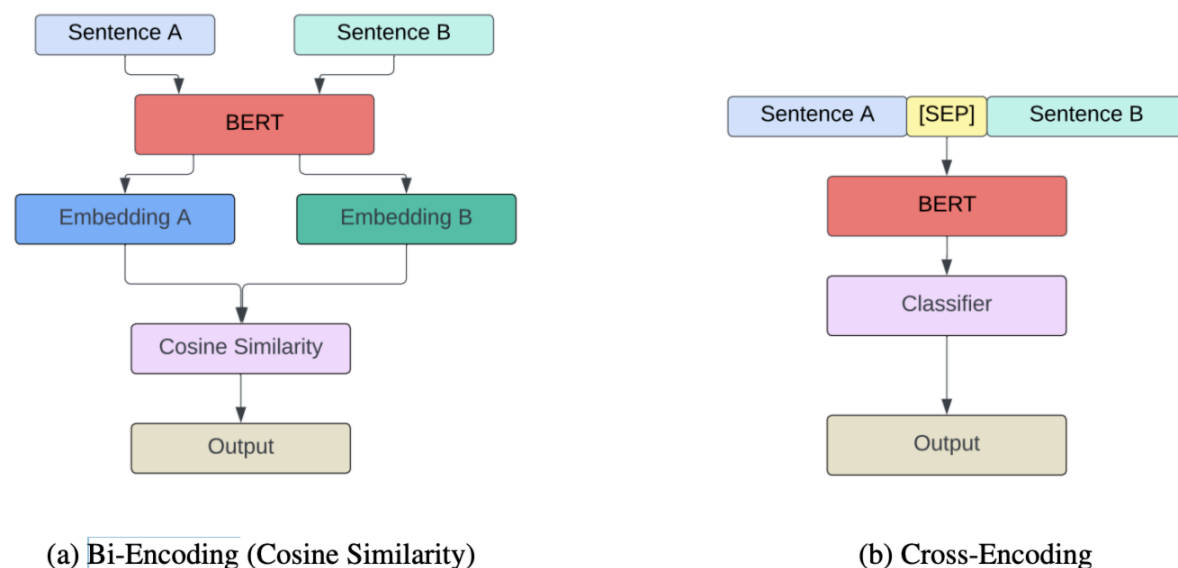
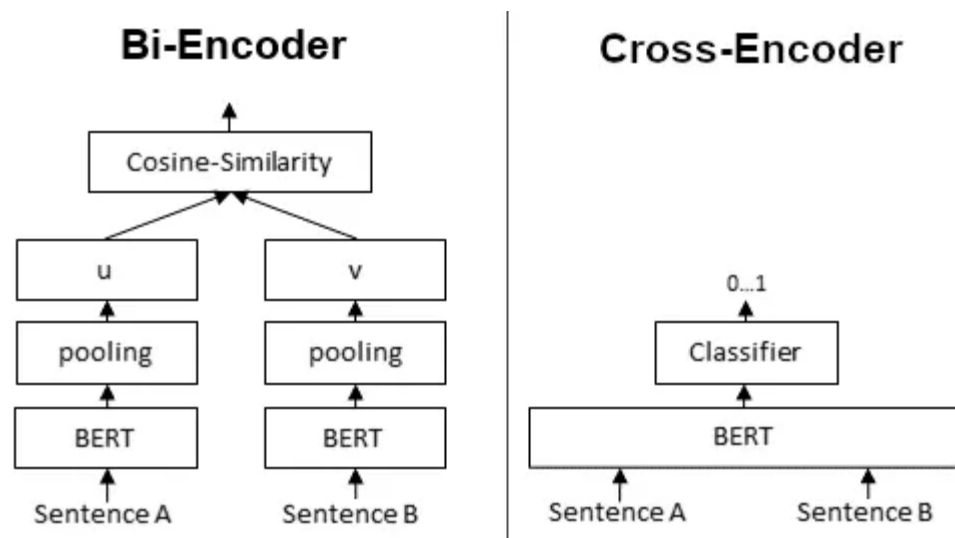


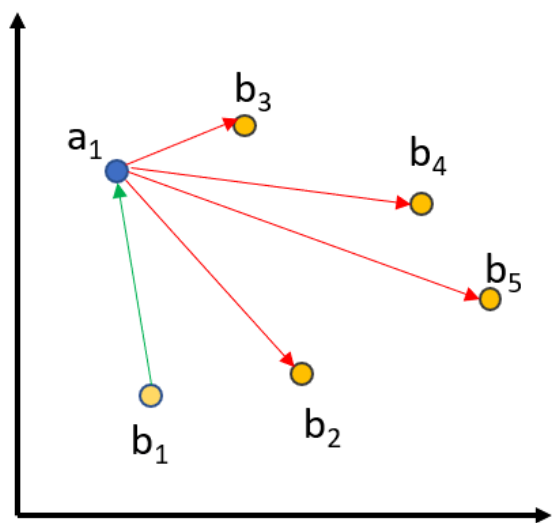
Figure 1: Bi-encoding vs. Cross-encoding

重排序模型通常是基于交叉编码器（Cross Encoder）架构实现的。它的核心思想是把两段文本（比如一个问题和一个文档，或者两个句子）放在一起处理，通过一个共享的神经网络，最终给出一个分数，表示这两段文本的相关性。这和另一种常见的模型——Sentence Transformers（也叫双编码器或嵌入模型）很不一样。



Sentence Transformers 是把每段文本单独转换成向量，然后通过计算向量之间的距离来判断相似性。而重排序模型则是让两段文本“相互关注”，一起通过网络处理。这种设计让重排序模型在判断文本相关性时更加精准，但代价是计算速度会慢一些。

重排序模型虽然计算速度慢，但它在某些场景下非常有用。比如，当你已经有了一个初步的搜索结果列表（比如通过 Sentence Transformers 搜索出来的），但这个列表可能包含很多相似的内容，这时候就需要重排序模型来“精挑细选”，找出最相关的结果。



一个用户提问，10w 个划分好的 chunk

步骤 1: chunk 检索，embedding 模型（轻量的模型）做嵌入，计算相似度

检索方法 1 -》10 个相关的 chunk

检索方法 2 -》 10 个相关的 chunk

检索方法 3 -》 10 个相关的 chunk

步骤 2: 检索结果合并，按照次序打分，合并后的 top10

步骤 3: reranker 模型（强大的模型） 对合并后的结果重新打分，重新排序， top3

步骤 4: 提问和步骤 3 汇总在提示词，调用大模型

# RAG 问答

## 问答流程

RAG 的主要实现流程包括**信息检索**和**大型语言模型调用**两个关键过程。

- 1. **问题理解：** 准确把握用户的意图，处理用户提问的模糊性、用词不规范等问题，将用户提问与知识库中的信息建立有效关联。
- 2. **知识检索：** 从外部知识库中检索与问题相关的知识。这需要解决用户提问与知识库多样化信息（如 PDF、PPT、Neo4j 等格式）之间的语义关联问题。
- 3. **答案生成：** 将检索结果与问题整合，利用大型语言模型生成最终的回答，确保答案与用户意图一致，同时保持自然、连贯的文本。此阶段还需应对大型模型输出可能存在的幻觉问题。

## 意图识别

意图识别允许系统更加灵活地适应用户的多样化需求。它允许系统在不同的上下文中识别用户意图，从而提供更准确、定制的回答。这种方法的优势在于通过使用专门的模型来处理特定领域的问题，可以提高系统的准确性和用户体验。

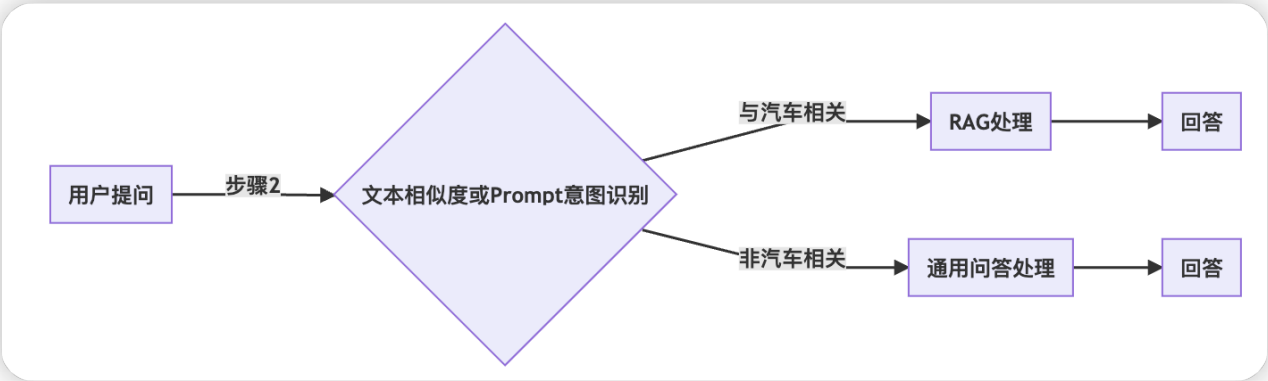
文本相似度

- 步骤 1：提取用户提问的嵌入向量
- 步骤 2：提取文档所有的嵌入向量
- 步骤 3：判断提问向量与文档向量的最低相似度，结合相似度大小进行判断

文本提示词

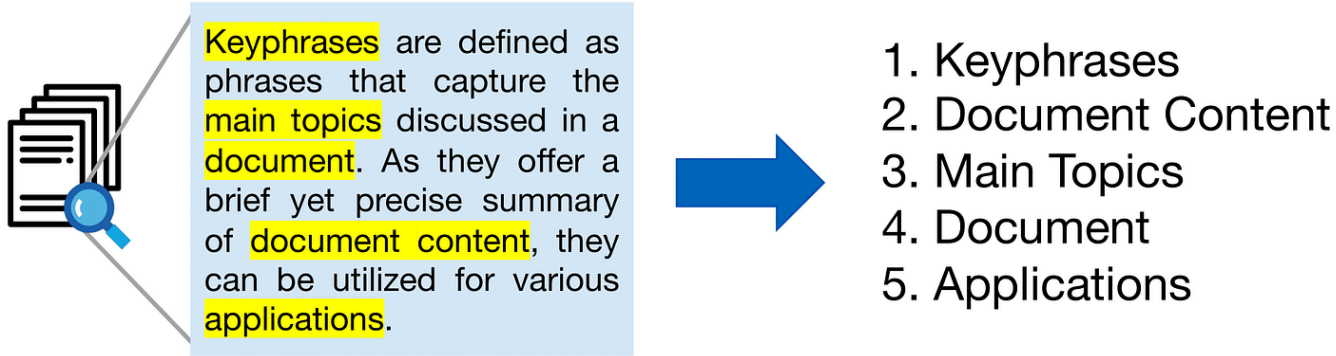
Shell

- 1 你是一个汽车维修和汽车销售的专家，请判断下面的提问是否与汽车使用相关。
- 2
- 3 {用户提问}
- 4
- 5 输出：相关 / 不相关



问答关键词

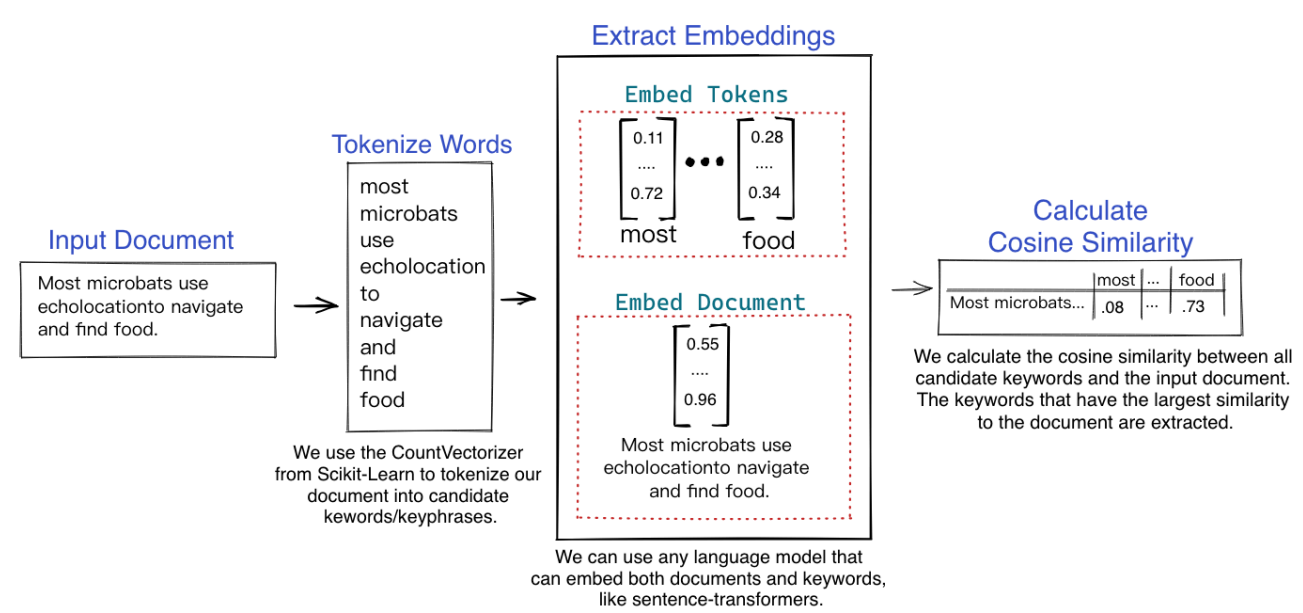
文本关键词抽取是自然语言处理领域的一项重要任务，其目标是从给定的文本中提取出最具代表性和有意义的单词或短语。这些关键词通常反映了文本的主题、内容或重要信息。常见的步骤包括分词、词性标注、停用词移除、计算词语权重以及关键词抽取算法等过程。



方法1：IDF

- 1. **分词 (Tokenization)**: 将文本拆分为单词或短语。这一步骤将文本转换为基本的语言单元，为后续的处理做准备。
- 2. **移除通用词 (Stopword Removal)**: 剔除常见的停用词，如"and"、"the"、"is"等，这些词在文本中普遍出现但往往没有实际的信息价值。这样做可以减少噪音，使关键词更集中在文本的内容性词汇上。
- 3. **计算逆文档频率 (IDF)**: 对于每个单词，计算其逆文档频率。逆文档频率是一个衡量单词重要性的指标，它通过对整个文本集中包含该词的文档数取倒数来计算。
- 4. **计算 TF-IDF 得分**: 对于每个单词，计算其 TF-IDF 得分，即词频 (TF) 与逆文档频率 (IDF) 的乘积。TF 表示单词在当前文档中的出现频率。
- 5. **排序和选取关键词**: 根据计算得到的 TF-IDF 得分对单词进行排序，选择排名前几的单词作为关键词。排名越高的单词表示在当前文档中具有更高的重要性。

方法2: KeyBERT



<https://github.com/MaartenGr/KeyBERT>

- 1. **Embedding 文本**: 首先，KEYBERT 使用预训练的 BERT 模型，例如 `distilbert-base-nli-mean-tokens`，将输入的文本嵌入到一个高维的向量空间中。BERT 模型能够学习丰富的语义表示，因此生成的向量能够捕捉文本的语义信息。
- 2. **计算余弦相似度**: 然后，KEYBERT 计算文档中每个候选关键词或关键短语与整个文档之间的余弦相似度。余弦相似度是一种衡量两个向量之间夹角的度量，它在这里用于度量嵌入向量之间的相似性。

3. **排序关键词：**最后，根据计算得到的余弦相似度值，KEYBERT 将关键词或关键短语排序，从而形成最终的关键词列表。余弦相似度越高，表示关键词与文档的语义相似度越大，因此在排序中位置越靠前。

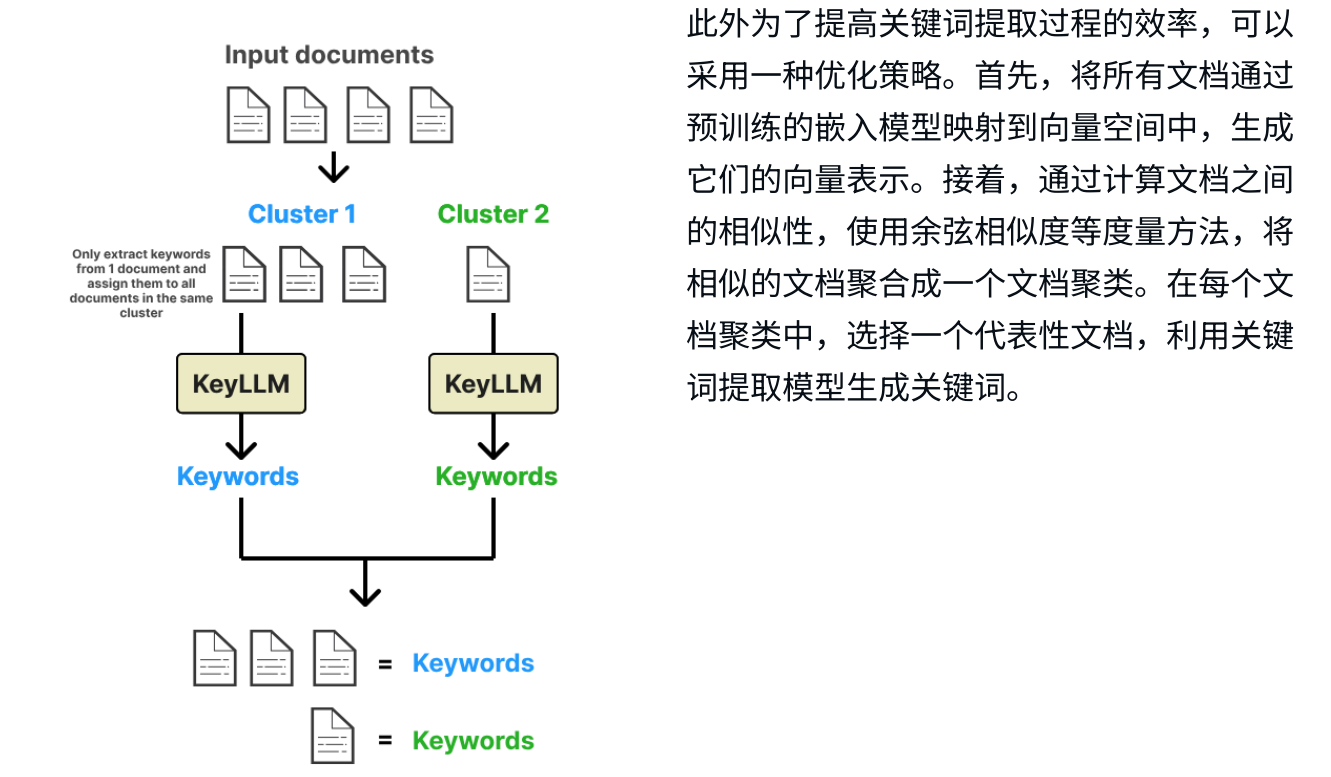
方法3：Prompt关键词提取

Plain Text

1 你是一个专业的文本理解专家，现在请你识别下面内容中的关键词，将关键词使用空格隔开：

2

3 {输入文本}



RAG 进阶

文档解析

# 文本切分

RAG 技术中划分 chunk 是为了更好地适应大模型的处理能力，提高检索和生成的效率和准确性，以及优化内容的相关性。

- 从大模型输入角度

**模型在预训练过程中通常有上下文长度限制**，如果输入的文本超过这个长度，超出部分会被丢弃，从而影响模型的性能表现。因此，需要将长文档分割成小块，以适应模型的上下文窗口。

- 从语义表示的差异性角度

**长文档中各个片段的语义可能存在较大差异**，如果将整个文档作为一个整体进行知识检索，会导致语义杂糅，影响检索效果。将长文档切分成多个小块，可以使得每个小块内部表意一致，块之间表意存在多样性，从而更充分地发挥知识检索的作用。

## 评价 Chunk 划分方法？

- 分块归因（Chunk Attribution）

分块归因用于评估每个检索到的分块是否对模型的响应产生了影响。它使用一个二元指标，**将每个分块归类为“有影响”（Attributed）或“无影响”（Not Attributed）**。分块归因与分块利用率（Chunk Utilization）密切相关，其中归因确定分块是否对响应产生了影响，而利用率衡量分块文本在影响中所占的程度。只有被归为“有影响”的分块才能有大于零的利用率得分。

- 分块利用率（Chunk Utilization）

分块利用率衡量每个检索到的分块中有多少文本对模型的响应产生了影响。这个指标的范围是 0 到 1，其中 1 表示整个分块影响了响应，而较低的值，如 0.5，则表示存在“无关”文本，这部分文本并未对响应产生影响。分块利用率与分块归因紧密相关，归因确定分块是否影响了响应，而利用率衡量分块文本在影响中所占的部分。只有被归为“有影响”的分块才能有大于零的利用率得分。

## 划分 Chunk 的注意事项

在进行 chunk 划分时，需要保留每个 chunk 的原始内容来源信息，这包括但不限于：

- **页面编号**：记录每个 chunk 来自文档的哪一页，有助于在需要时快速定位原始信息。
- **文档标识**：为每个文档分配一个唯一的标识符，以便在检索时能够准确引用。
- **版本控制**：如果文档会更新，记录 chunk 对应的文档版本，确保内容的一致性和准确性。

随着时间的推移，原始文档可能会更新或修改（可能使用新的文档处理方法重新划分 chunk）。因此，在划分 chunk 时需要考虑：

- **更新策略**：制定一个清晰的策略，以确定何时以及如何更新 chunk，以保持信息的最新性。
- **版本兼容性**：确保新旧版本的 chunk 能够兼容，或者能够明确区分不同版本的 chunk。

此外 chunk 之间的顺序可能对理解整个文档至关重要：

- **顺序标记**：为每个 chunk 分配顺序编号或时间戳，以保持它们的逻辑顺序。
- **顺序检索**：在检索时，确保能够按照正确的顺序检索和展示 chunk。

虽然存在不同的 chunk 划分方法，但文本内容的复杂性和多样性要求我们在划分 chunk 时采用灵活和多样的方法：

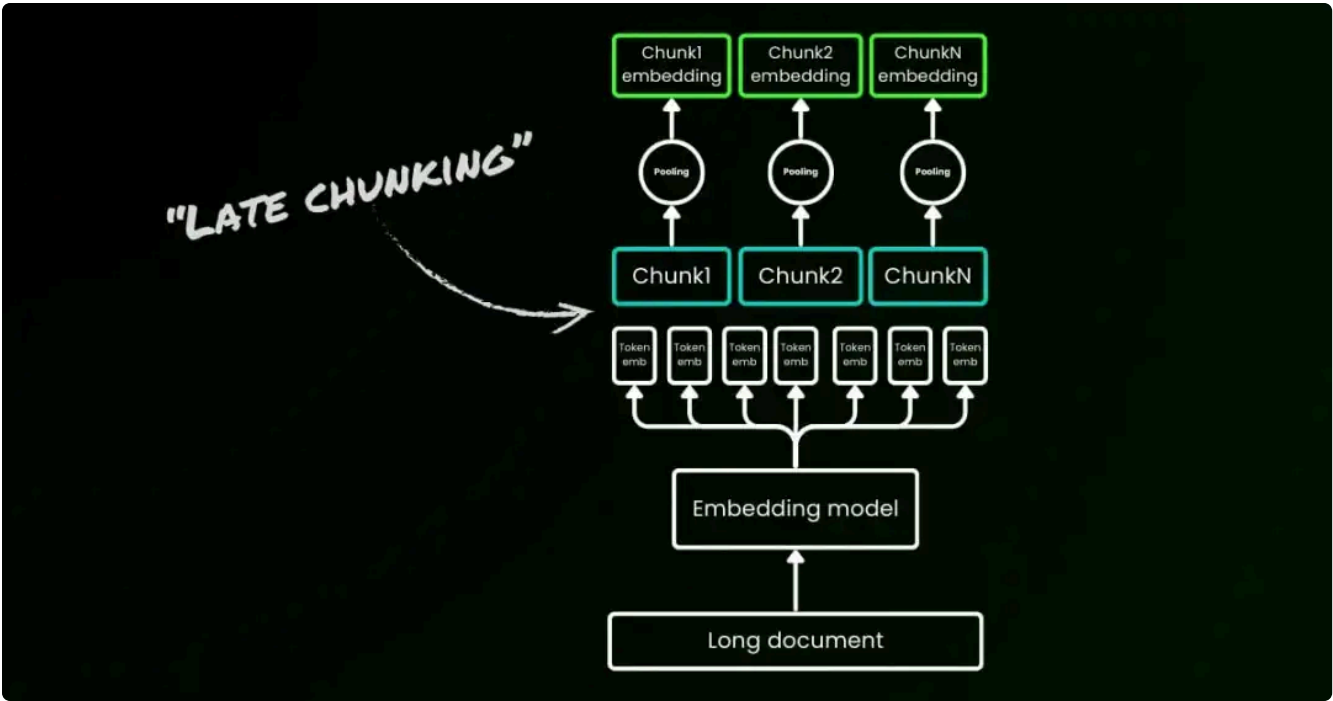
- **多种划分方法**：根据文本的特点和需求，可能需要结合固定大小、滑动窗口、语义分块等多种方法。
- **上下文感知**：在划分 chunk 时，考虑上下文的连贯性，避免将语义上紧密相关的信息分割开。

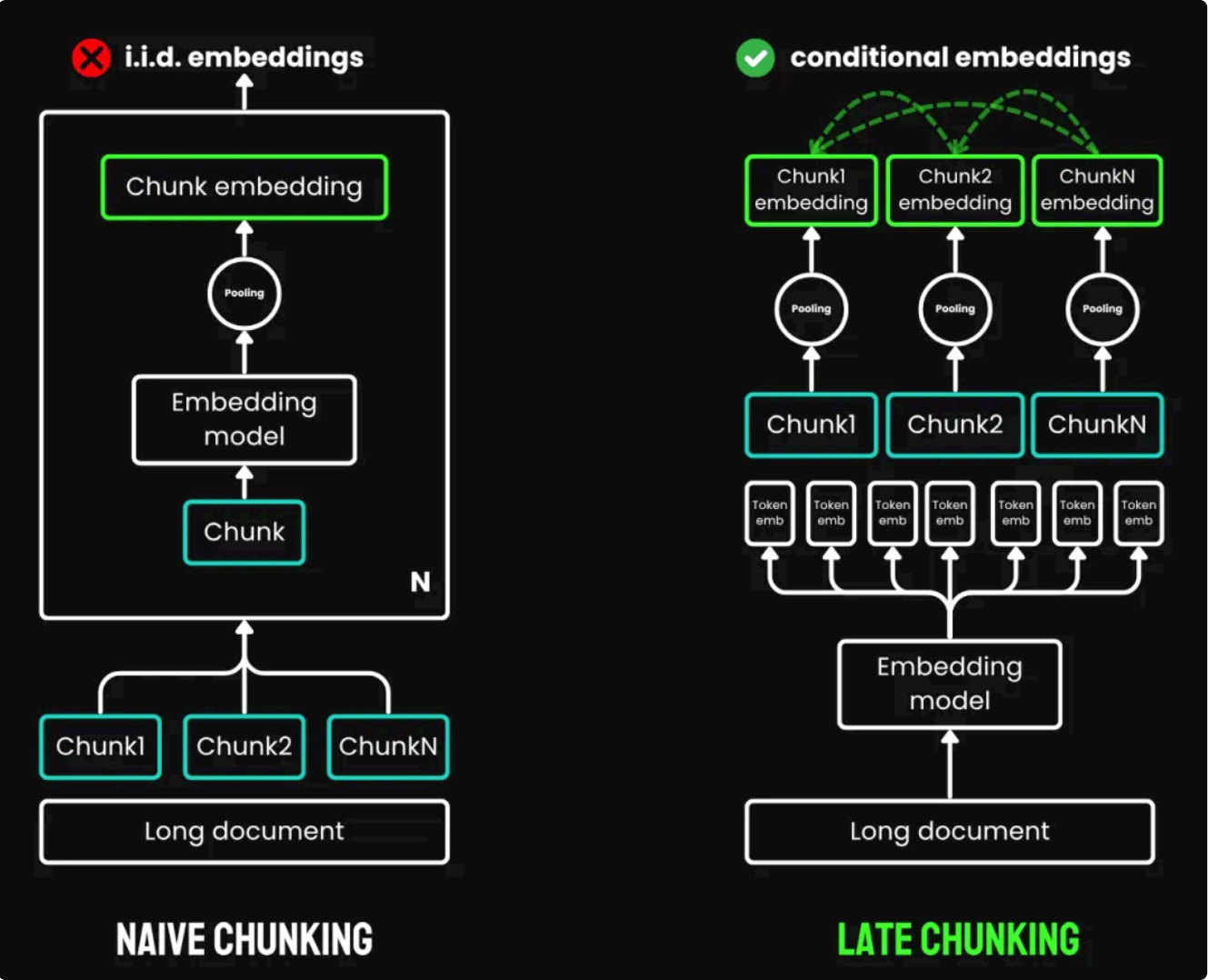
## 进阶划分方法

- <https://jina.ai/news/late-chunking-in-long-context-embedding-models/>
- <https://jina.ai/news/finding-optimal-breakpoints-in-long-documents-using-small-language-models/>

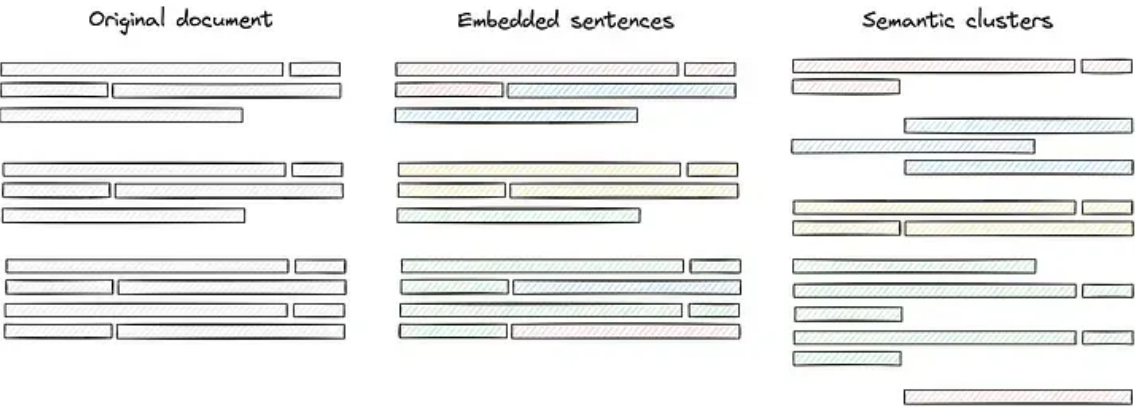
| 划分方法                              | 实现过程                                                                                                                                                                                                                                                                                                                       | 优缺点              |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 固定大小块分块策略（Fixed Size Chunking）    | 将文本分成固定大小的块，适用于内容格式和大小相似的数据集。                                                                                                                                                                                                                                                                                              | 快，粗暴，丢失格式        |
| 滑动窗口分块策略（Sliding Windows）         | 新的块与前一个块的内容重叠，包含部分内容，以更好地捕捉每个块周围的上下文。                                                                                                                                                                                                                                                                                      | 解决文本被截断，产生无意义的块。 |
| 上下文感知分块策略（Context-aware Chunking） | 包括基于 token、句子、正则表达式和 Markdown 的分块技术。                                                                                                                                                                                                                                                                                       |                  |
| 面向主题的分块技术                         | 使用句子嵌入来识别文档中主题的变化，确保每个块封装一个单一的、连贯的主题。                                                                                                                                                                                                                                                                                      |                  |
| 语义分块（Semantic Chunking）           | <ul style="list-style-type: none"><li>核心目的是把相同语义的 token 聚集在一起，不同语义的 token 分开，利于后续的检索和重排。</li><li>通过计算句子组合的相似度来识别局部极小值，这表明潜在的话题转换，可以用阈值来确定重要的边界。</li></ul>                                                                                                                                                                  |                  |
| 递归分块（Recursive Chunking）          | <ul style="list-style-type: none"><li>使用一组分隔符，以分层和迭代的方式将文本划分为更小的块。</li><li>如果最初分割文本的尝试没有产生所需大小的块，则该方法会使用不同的分隔符递归地调用结果块，直到达到所需的块大小。</li></ul>                                                                                                                                                                               |                  |
| 基于大模型的分块（LLM Based Chunking）      | <ul style="list-style-type: none"><li>通过大模型来理解文本，并将文本进行拆分为句子或段落。</li><li>依赖大模型能力，可以拆分原始文本，也可以从原始文本中生成新的文本</li></ul>                                                                                                                                                                                                        |                  |
| Late Chunking                     | <ol style="list-style-type: none"><li>首先将整个文本输入到长上下文嵌入模型中，生成每个 token 的向量表示。这意味着模型能够捕捉到整个文本的上下文信息，而不仅仅是单个分块的局部上下文。</li><li>在获得整个文本的 token 级嵌入后，再将文本分割成多个分块。每个分块包含一定数量的 token，并且可能存在重叠部分，以保留边界处的上下文信息。</li><li>对于每个分块，使用 mean pooling 或其他池化方法，将分块内所有 token 的向量表示聚合成一个分块嵌入。由于这些分块嵌入是基于整个文本的上下文生成的，因此它们能够保留更多的上下文信息。</li></ol> |                  |

提示词模版 = “你是一个文本切分专家，对下面的文本进行组织，并划分为不同的句子。  
「原始页面的内容」”





语义分块

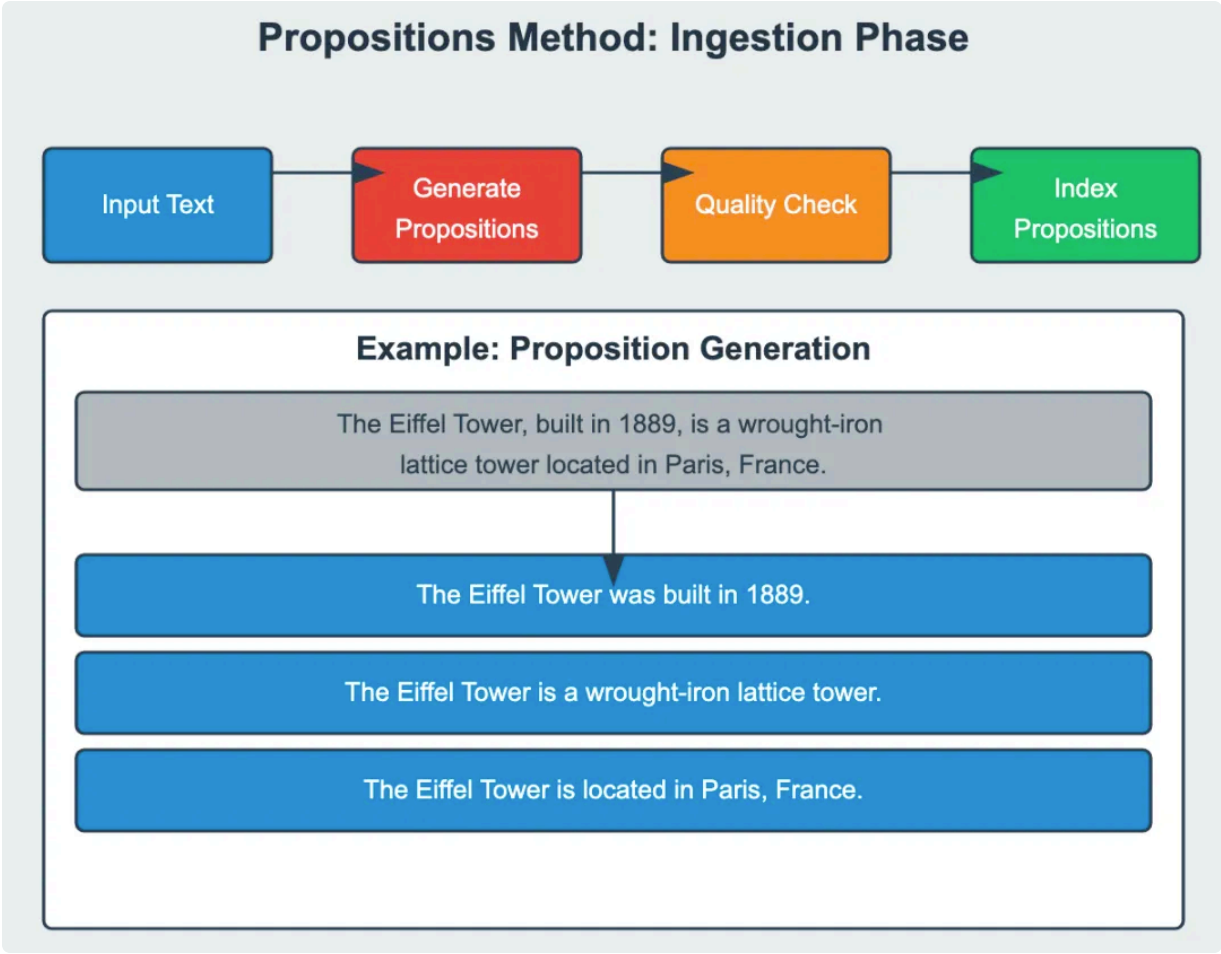


传统文本分割方法通常在任意点分割文档，可能会破坏信息和上下文的连贯性。语义分块通过尝试在更自然的断点处分割文本，解决了这一问题，从而在每个块内保持语义连贯性。

- **文档预处理：** 读取 PDF 并将其转换为字符串。

- 语义分块：利用 LangChain 的 SemanticChunker 和 OpenAI 嵌入向量。提供了三种断点类型：
  - 'percentile': 在大于 X 百分位数的差异处分割。
  - 'standard\_deviation': 在大于 X 标准差的差异处分割。
  - 'interquartile': 使用四分位数距离来确定分割点。在此实现中，使用了'percentile'方法，阈值为 90。
- **向量存储创建**：使用 OpenAI 嵌入向量创建语义块的向量表示，并从这些嵌入向量创建 FAISS 向量存储。
- **检索器设置**：配置检索器以获取给定查询的最相关的 2 个块。

命题分块



命题分块的方法将输入文本分解为原子的、事实性的、自包含的且简洁的命题，并将这些命题编码到向量存储中，以便后续检索使用。

命题分块方法的动机是构建一个系统，将文本文档分解为简洁的事实性命题，以便进行更细致和精确的信息检索。使用命题可以更好地处理特定查询，尤其是在从详细或复杂的文本中提取

知识时。

- **文档分块**：使用 RecursiveCharacterTextSplitter 将输入文档分割成较小的块，以便 LLM 处理。
- **命题生成**：使用 LLM（例如"llama-3.1-70b-versatile"）从每个块中生成命题。输出为一系列事实性、自包含的陈述，无需额外上下文即可理解。
- **质量检查**：使用第二个 LLM 根据准确性、清晰性、完整性和简洁性对命题进行评分。满足所有类别所需阈值的命题将被保留。
- **嵌入命题**：通过 OllamaEmbeddings 模型将通过质量检查的命题嵌入到向量存储中，以便在查询时进行基于相似性的检索。
- **检索和比较**：构建两个检索系统：一个使用基于命题的块，另一个使用较大的文档块。两者都通过多个查询进行测试，以比较它们的性能和返回结果的精确度。

## 内容增强

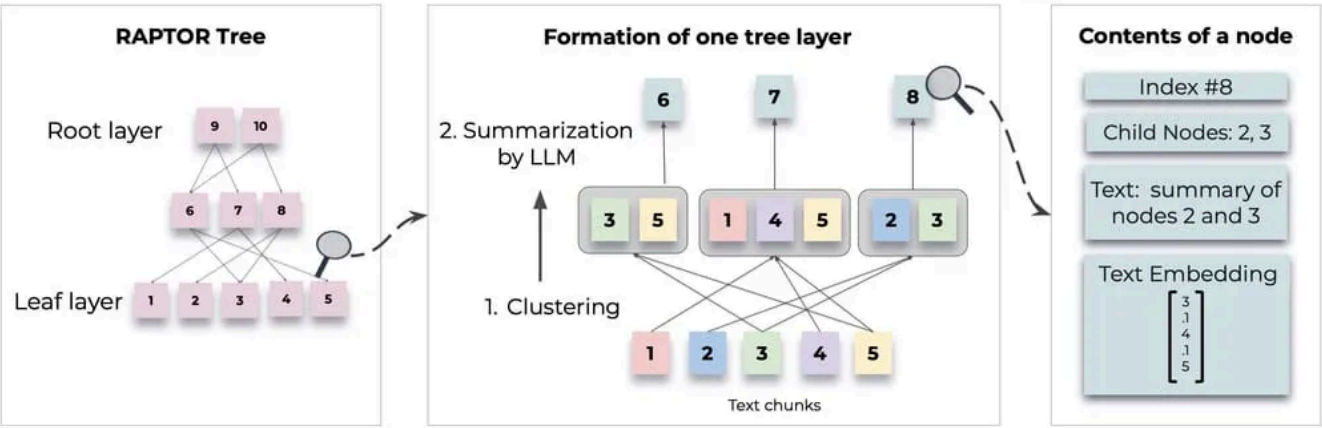
### 提问生成（知识库的提问增强） -》FAQ 问答

生成相关问题通过为每个文本片段生成各种相关问题，并将这些问题纳入向量数据库中，从而增强了标准的检索过程，提高了找到可作为生成式问答上下文的相关文档的可能性。

- **PDF 处理和文本分块**：处理 PDF 文档并将其分割成可管理的文本片段。
- **问题增强**：使用 OpenAI 的语言模型在文档和片段级别生成相关问题。
- **向量存储创建**：使用 OpenAI 的嵌入模型计算文档嵌入，并创建 FAISS 向量存储。
- **检索和答案生成**：使用 FAISS 找到最相关的文档，并基于提供的上下文生成答案。

### RAPTOR（推荐）

RAPTOR（Recursive Abstractive Processing and Thematic Organization for Retrieval）结合了层次化文档摘要、基于嵌入的检索和上下文答案生成。**RAPTOR 旨在通过创建摘要的多层次树结构**，高效地处理大型文档集合，从而实现从宽泛到详细的信息检索。



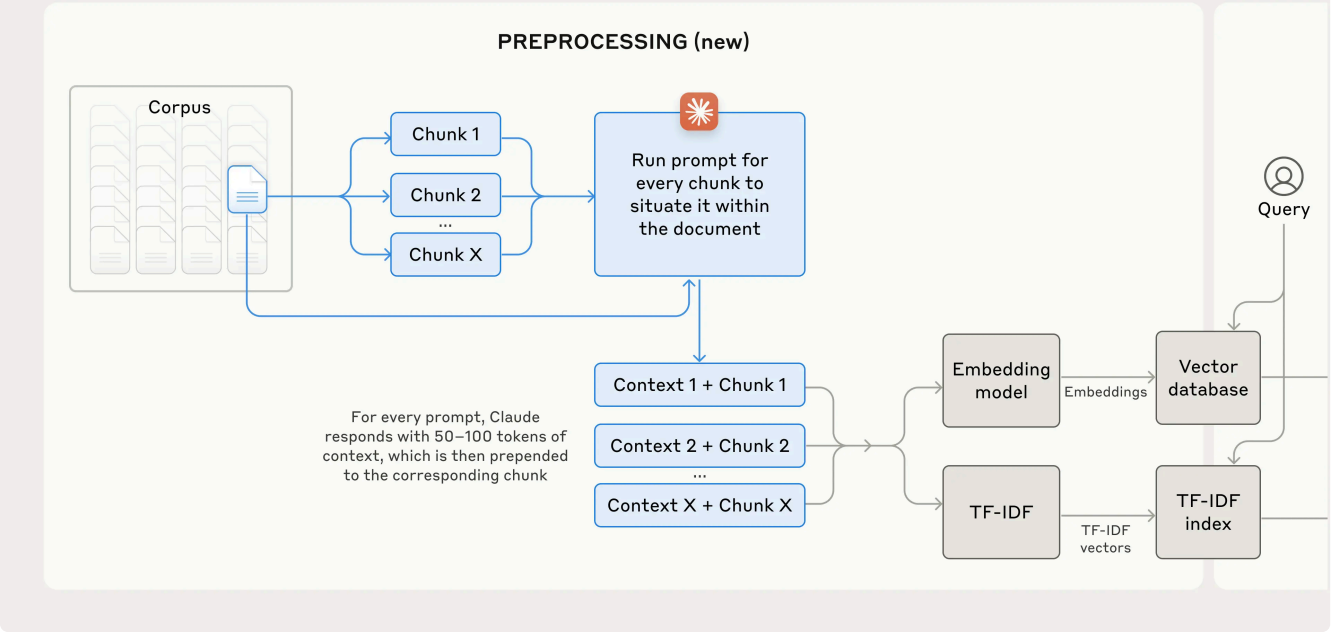
- **树构建：**从原始文档开始，逐层嵌入文本，使用高斯混合模型等聚类算法对嵌入进行聚类，并为每个聚类生成摘要，直到达到单个摘要或最大层数。
- **嵌入和检索：**将所有文档和摘要的嵌入存储在向量存储中（如 FAISS），以便进行高效的相似性搜索。对于给定查询，嵌入查询并检索最相似的文档 / 摘要。
- **上下文压缩：**使用语言模型从检索到的文档 / 摘要中提取与给定查询最相关的部分。
- **答案生成：**将相关部分组合成上下文，并使用语言模型基于此上下文和原始查询生成答案。

## Contextual Chunk Headers

<https://www.anthropic.com/news/contextual-retrieval>

上下文块头（Contextual Chunk Headers，简称 CCH）通过创建包含更高层次上下文（如文档级或节级上下文）的块头，并在嵌入之前将这些块头添加到块的前面。这使得嵌入能够更准确和完整地表示文本的内容和含义。在测试中，该功能显著提高了检索质量。除了提高正确信息检索的速率外，CCH 还降低了搜索结果中不相关结果出现的速率。

# Contextual Retrieval Preprocessing



通过在块头前面添加更高层次的上下文来增强块。这个块头可以简单到只是文档标题，也可以使用文档标题、简洁的文档摘要以及节和子节标题的完整层次结构的组合。

## 提问改写（Query transformation）

用户输入的问题可能表述模糊、不明确，这会导致检索系统无法准确理解用户意图，从而无法检索到有价值的信息。此外，用户输入的复杂性增加，如自然对话、多轮对话数据，使得问题形式更加多元，紧密关联上下文，输入风格更加口语化，这也增加了检索的难度。

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Routing Strategy               | <p>Routing Strategy（路由策略）是一种根据用户查询选择合适的工具或工具组合的方法。这种策略可以使用不同的选择器（selectors）来实现，它们可以是基于大型语言模型（LLM）的选择器，也可以是基于函数调用的选择器。选择器可以是单选（single selection）或多选（multi-selection）。</p> <ul style="list-style-type: none"><li>Pydantic 单选</li><li>Pydantic 多选</li><li>LLM 单选</li><li>LLM 多选</li></ul>                                                                                                                                                        |
| Query Rewriting                | <p>Query Rewriting（查询重写）策略涉及将单个查询改写为多个查询，以便对检索器执行这些查询。这是高级检索技术中的关键步骤。通过查询重写，可以为[ensemble retrieval]和[fusion]生成多个查询，从而提高检索结果的质量。</p> <div><div>Plain Text</div><pre>1 query_gen_str = "" 2 You are a helpful assistant that generates multiple search queries based on   a \single input query. 3 Generate {num_queries} search queries, one on each line, \related to the foll   owing input query: 4 Query: {query}Queries: 5 ""</pre></div> |
| Multi-Query Rewriting          | <p>Multi-Query Rewriting 策略的核心思想是通过从不同角度生成多个查询来提高检索召回率。这种方法特别适合于以下场景：</p> <ul style="list-style-type: none"><li>用户查询不清晰或含糊；</li><li>需要从多个角度理解用户意图；</li><li>单一查询无法覆盖完整信息。</li></ul> <p>从用户的原始查询中生成多个不同的查询变体，以覆盖更广泛的信息和视角。这可以通过使用语言模型（LLM）来实现，模型会根据原始查询生成多个相关但不同的查询。</p>                                                                                                                                                                        |
| Problem Decomposition Strategy | <p>Problem Decomposition Strategy（问题分解策略）是一种针对复杂查询的处理方法，它通过将复杂问题分解成多个更简单的子问题来提高检索的准确性和效率。这种方法可以：</p> <ul style="list-style-type: none"><li>提高复杂问题的检索准确性；</li><li>使检索过程更有条理；</li><li>便于处理多步骤推理问题。</li></ul>                                                                                                                                                                                                                                    |

|                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Step-Back Strategy</b>                      | <p>Step-Back Strategy（后退一步策略）是一种通过从更抽象或更基础的层面重新审视问题的方法。这种策略可以：</p> <ul style="list-style-type: none"><li>• 扩大检索范围；</li><li>• 获取更完整的上下文信息；</li><li>• 提供更全面的答案。</li></ul> <p>首先，让 LLM 根据原始问题提出一个更高层次的后退问题，并检索这个后退问题的相关信息。基于高层次概念或原则的事实，LLM 可以推理出原始问题的解决方案。</p>                                                                                                                                                                                                                                                                                      |
| <b>Hypothetical Document Embeddings (HyDE)</b> | <p>HyDE 不是直接根据原始查询搜索相关文档，而是首先构建一个可能回答查询的假设性文档。虽然这个假设性文档在细节上可能不是完全准确的，但它作为一个相关文档的有价值示例，捕捉到了相关性的本质。</p> <p>检索过程利用对比训练中编码的文档 - 文档相似性，通过内积识别与假设答案紧密对齐的文档。最相似的真实文档被检索出来，并作为查询的潜在响应呈现，提高了检索的准确性。</p> <div><h3>Hypothetical Document Embeddings</h3><div><div><p>Standard</p><div><div>Question</div><div>Question is used to Search Docs</div><div>Search Results</div></div></div><div><p>HyDE</p><div><div>Question</div><div>LLM Answers the Question</div><div>Answer is used to Search Docs</div><div>Search Results</div></div></div></div></div> |

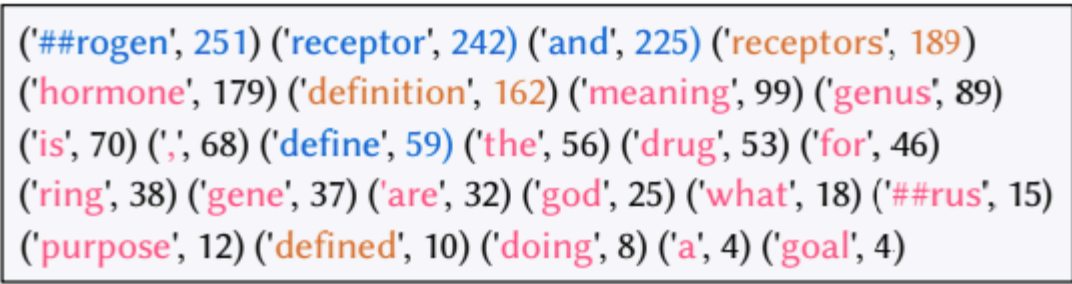
编码与重排序

BERT 稀疏表示

SPLADE（Sparse Lexical and Expansion Model for First Stage Ranking）是一种用于信息检索第一阶段排名的模型，它旨在通过学习文档和查询的稀疏表示来提高检索效率和效果。

SPLADE 模型的核心思想是学习文档和查询的稀疏表示，这样可以利用倒排索引进行高效的检

索。稀疏表示意味着在表示中只有少数词汇具有非零权重，这有助于减少计算量并提高检索速度。



**Figure 1: An example of the SPLADE encoding for the query androgen receptor define. Each query term is represented as a (‘token’, weight) tuple, with blue terms representing the original input query terms, orange terms representing alternate inflections on those original terms, and pink terms representing entirely new terms. Figure reproduced with permission from Mackenzie et al. [16].**

## Qwen3-Embedding

<https://qwenlm.github.io/blog/qwen3-embedding/>

Qwen3 嵌入模型系列是 Qwen 家族的最新专有模型，专门用于文本嵌入和排序任务。该系列基于 Qwen3 系列的密集基础模型构建，提供了多种尺寸（0.6B、4B 和 8B）的全面文本嵌入和重排序模型。它继承了基础模型卓越的多语言能力、长文本理解和推理技能。Qwen3 嵌入系列在多种文本嵌入和排序任务中取得了显著进展，包括文本检索、代码检索、文本分类、文本聚类 and 双语挖掘。

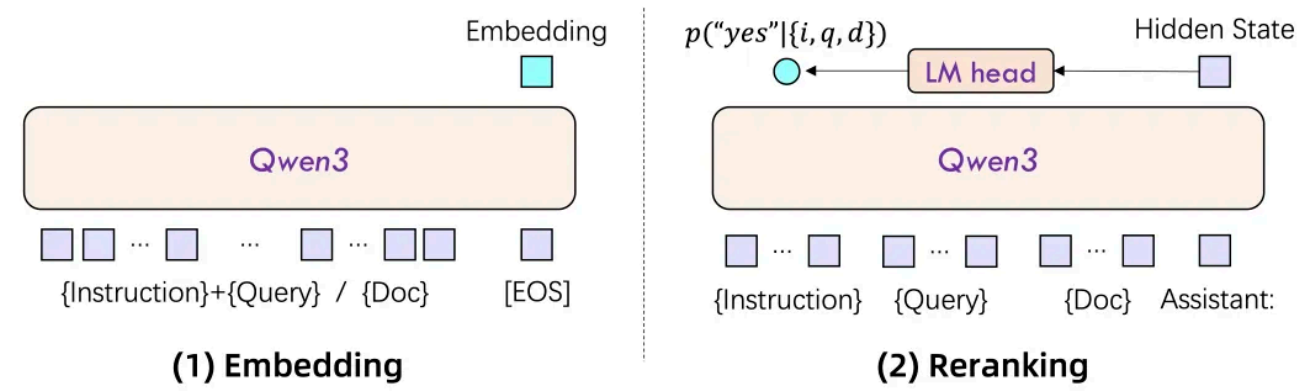
Qwen3 嵌入系列提供了从 0.6B 到 8B 的全系列尺寸，涵盖嵌入和重排序模型，满足不同用例对效率和效果的需求。得益于 Qwen3 模型的多语言能力，Qwen3 嵌入系列支持 100 多种语言，包括各种编程语言，提供强大的多语言、跨语言和代码检索能力。

|                                          | Qwen3-<br>Embedding-8B | Qwen3-<br>Embedding-4B | Qwen3-<br>Embedding-0.6B | Gemini<br>Embedding | Cohere-embed-<br>multilingual-<br>v3.0 | text-<br>embedding-3-<br>large | multilingual-<br>e5-large-<br>instruct | gte-Qwen2-<br>7B-instruct |
|------------------------------------------|------------------------|------------------------|--------------------------|---------------------|----------------------------------------|--------------------------------|----------------------------------------|---------------------------|
| MMTEB<br><small>Mean-Task</small>        | 70.58                  | 69.45                  | 64.33                    | 68.37               | 61.12                                  | 58.93                          | 63.22                                  | 62.51                     |
| MTEB (en v2)<br><small>Mean-Task</small> | 75.22                  | 74.60                  | 70.70                    | 73.30               | 66.01                                  | 66.43                          | 65.53                                  | 70.72                     |
| MTEB-Code                                | 80.68                  | 80.06                  | 75.41                    | 74.66               | 51.94                                  | 58.95                          | 65.00                                  | 56.41                     |

| 模型类型  | 模型名称                 | 尺寸   | 层数 | 序列长度 | 嵌入维度 | 支持 MRL | 指令感知 |
|-------|----------------------|------|----|------|------|--------|------|
| 文本嵌入  | Qwen3-Embedding-0.6B | 0.6B | 28 | 32K  | 1024 | 是      | 是    |
| 文本嵌入  | Qwen3-Embedding-4B   | 4B   | 36 | 32K  | 2560 | 是      | 是    |
| 文本嵌入  | Qwen3-Embedding-8B   | 8B   | 36 | 32K  | 4096 | 是      | 是    |
| 文本重排序 | Qwen3-Reranker-0.6B  | 0.6B | 28 | 32K  | -    | -      | 是    |
| 文本重排序 | Qwen3-Reranker-4B    | 4B   | 36 | 32K  | -    | -      | 是    |
| 文本重排序 | Qwen3-Reranker-8B    | 8B   | 36 | 32K  | -    | -      | 是    |

- **MRL 支持**：表示嵌入模型是否支持自定义最终嵌入的维度。
- **指令感知**：表示嵌入或重排序模型是否支持根据不同任务自定义输入指令。
- 评估表明，对于大多数下游任务，使用指令（instruct）通常比不使用指令能提高 1% 到 5% 的性能。

嵌入模型和重排序模型的核心思想是以任务感知的方式评估相关性。给定一个查询 q 和一个文档 d，嵌入和重排序模型根据指令 l 定义的相似性标准来评估它们的相关性。



- 对于文本嵌入，使用带有因果注意力的大型语言模型（LLM），并在输入序列的末尾附加一个 [EOS] 标记。最终嵌入是从最后一层对应于 [EOS] 标记的隐藏状态中得出的。为了确保在下游任务中嵌入能够遵循指令，将指令和查询拼接为一个单一的输入上下文，而文档在处理之前保持不变。查询的输入格式如下：

Shell

```
1 {指令} {查询}<|endoftext|>
```

- 对于重排序，使用 LLM 进行单个上下文内的逐点重排序。与嵌入模型类似，为了实现指令遵循能力，在输入上下文中包含指令。我们使用 LLM 的聊天模板，并将相似性评估任务视为一个二分类问题。输入到 LLM 的模板如下所示：

Shell

```
1 <|im_start|>system
2 根据提供的查询和指令，判断文档是否符合要求。注意，答案只能是“是”或“否”。<|im_end|>
3 <|im_start|>user
4 <指令>: {指令}
5 <查询>: {查询}
6 <文档>: {文档}<|im_end|>
7 <|im_start|>assistant
8 <think>
9
10 </think>
```

## 联网搜索（默认不打开）

RAG 可以利用搜索引擎作为其检索组件的一部分，从互联网上获取广泛的知识。搜索引擎提供的强大索引和检索能力，是 RAG 能够获取外部信息的基础。RAG 将搜索引擎的检索结果作为“原材料”，再通过 LLM 进行加工和提炼，最终生成更高级的答案。

- 智谱 Web Search（国内、安全审核过滤）：<https://open.bigmodel.cn/dev/howuse/websearch>
- SerpAPI Google Search API：<https://serpapi.com/>
- Serper Google Search API：<https://serper.dev/>

- 网页内容读取 / jina.ai
  - <https://jina.ai/reader/>
  - <https://github.com/jina-ai/reader>
  - <https://huggingface.co/jinaai/ReaderLM-v2>

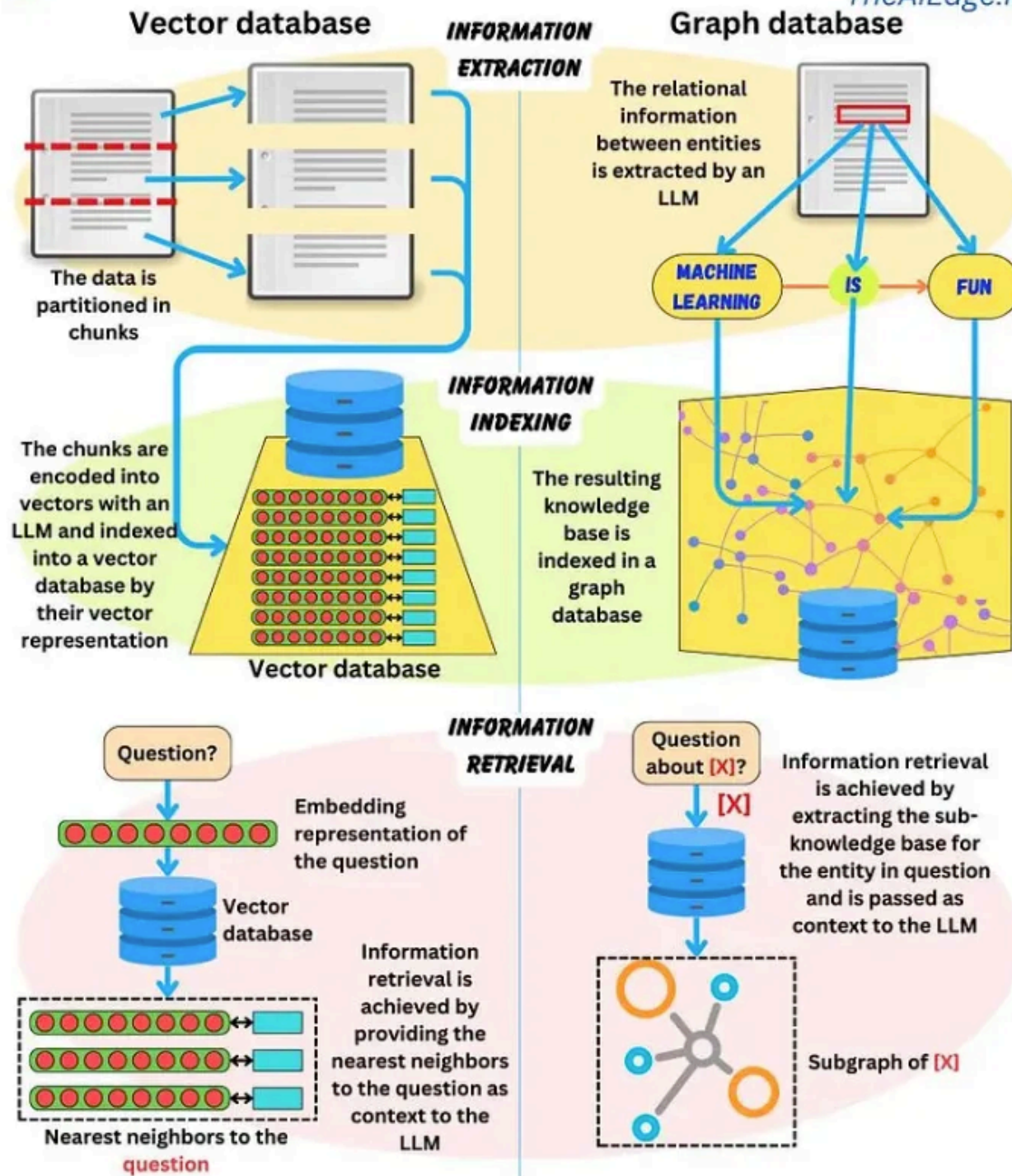
## GraphRAG（图谱构建、图谱问答）

与标准 RAG 模型不同，GraphRAG 擅长识别不同信息片段之间的联系并从中提取见解。这使其成为需要从大型数据集或难以总结的文档中提取见解的用户的理想选择。

三国演义 -> 历史人物，个人介绍，职位 -> 构建三国演义的知识图谱 -> 基于图谱问答

# Vector Database Vs Graph Database for RAG

TheAiEdge.io



- 索引阶段

- **文本分块:** 将源文本分割成可管理的块。
- **元素提取:** 使用 LLMs 识别实体和关系。

- **图构建**：从提取的元素构建图。
  - 实体消歧、实体合并
- **社区检测**：应用 Leiden 等算法查找社区。
- **社区摘要**：为每个社区创建摘要。
- 查询阶段
  - **本地答案生成**：使用社区摘要生成初步答案。
  - **全局答案综合**：结合本地答案形成全面响应。

<https://arxiv.org/pdf/2506.05690>

When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation

<https://arxiv.org/pdf/2506.13782>

XGraphRAG: Interactive Visual Analysis for Graph-based Retrieval-Augmented Generation

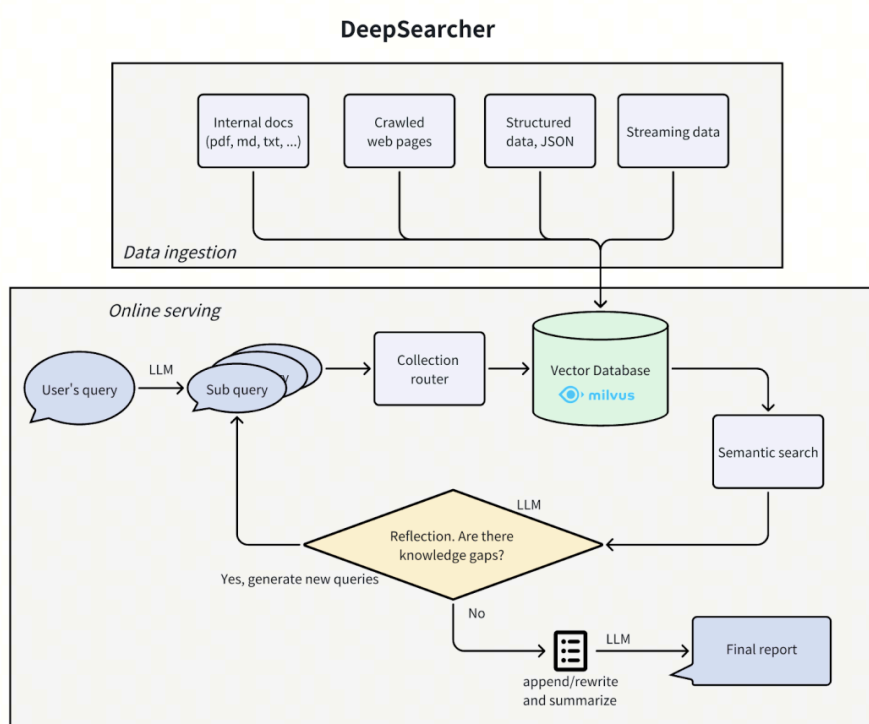
## Deep Research

Deep Research 是 OpenAI 开发的一种新工具，它可以在短时间内（5 分钟 - 几个小时）完成复杂的互联网研究任务，通常这些任务需要人类花费数小时甚至更长时间。**它通过分析和整合数百个在线来源的信息，生成一份像专业分析师那样全面的报告。**

<https://openai.com/index/introducing-deep-research/>

<https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research>

Deep Research 的输出会附带清晰的引用和思路总结，方便用户参考和验证信息。Deep Research 特别擅长找到那些需要浏览多个网站才能发现的冷门、非直观信息，从而节省用户宝贵的时间。通过训练，Deep Research 学会了如何规划并执行一系列步骤来找到需要的数据。**如果遇到问题，它还会根据实时信息调整方向，甚至回溯到之前的步骤。**此外，Deep Research 还能浏览用户上传的文件，使用 Python 工具绘制和修改图表，并将生成的图表或网页上的图像嵌入到回答中，同时引用来源中的具体句子或段落。这种训练方式让深度研究在解决现实问题的测试中达到了新的高度。



## 1. 搜索结果获取

应用通过配置好的 Google 自定义搜索或 Bing 搜索 API 获取指定搜索词的全面搜索结果。你可以选择使用 Google 或 Bing 的搜索服务，获取最相关的网页信息。

## 2. 内容提取

利用 JinaAI 技术，应用会提取并处理选定搜索结果的内容，确保信息的准确性和相关性。这一步是确保报告质量的关键环节，它会筛选出最有用的信息供后续处理。

## 3. 报告生成

在获取和处理好搜索结果后，应用会根据你选择的 AI 模型（比如 Gemini、GPT-4、Sonnet 等）生成详细的报告。报告内容会根据你的自定义提示进行深度整合和分析，提供有洞察力的输出。

|  | 擅长做什么         | 不擅长做什么               |
|--|---------------|----------------------|
|  | 低代码、搭建大模型的应用  | 复杂的应用（graphragwwear） |
|  | 简单的逻辑分支、判断    | 非文本的知识库问答            |
|  | 快速集成知识库、问答    | 不适合做元信息（原始的项目数据）存储   |
|  | 构建了流程，ui 快速生成 |                      |

## 大模型本地部署

- ollama： pc，简单实用，cpu 用的更多；
- sglang、vllm： linux，支持 gpu 加速，支持 kv 计算结果 缓存 -》速度更快，更稳定

## SGLang

<https://docs.sglang.ai/index.html>

数据隐私、速度

## Python

```
1 # To enable multi-GPU tensor parallelism, add --tp 2.
2 python -m sglang.launch_server --model-path meta-llama/Meta-Llama-3-8B-Instruct --tp 2
3
4 # To enable multi-GPU data parallelism, add --dp 2.
5 # Data parallelism is better for throughput if there is enough memory.
6 python -m sglang_router.launch_server --model-path meta-llama/Meta-Llama-3-8B-Instruct --dp 2 --tp 2
7
8 # If you see out-of-memory errors during serving
9 # try to reduce the memory usage of the KV cache pool by setting a smaller value of --mem-fraction-static.
10 python -m sglang.launch_server --model-path meta-llama/Meta-Llama-3-8B-Instruct --mem-fraction-static 0.7
11
12 # To run tensor parallelism on multiple nodes, add --nnodes 2.
13 # If you have two nodes with two GPUs on each node and want to run TP=4, let sgl-dev-0 be the hostname of the first node and 50000 be an available port
14 # Node 0
15 python -m sglang.launch_server --model-path meta-llama/Meta-Llama-3-8B-Instruct --tp 4 --dist-init-addr sgl-dev-0:50000 --nnodes 2 --node-rank 0
16
17 # Node 1
18 python -m sglang.launch_server --model-path meta-llama/Meta-Llama-3-8B-Instruct --tp 4 --dist-init-addr sgl-dev-0:50000 --nnodes 2 --node-rank 1
```

---

## RAG 评价指标

---

## RAG 实践重难点

## RAG 数据集（不需要训练）是从哪儿来的？

知识库：公司内部用于回答相关问题的文档

用户提问：模拟生成，或采集用户真实的问题



### RAG 效果评价:

- 检索效果（判断检索到的 chunk 排序精度，ndcg）：通过用户的提问是否能检索到相关的 chunk / 页面位置。
- 生成效果（[https://blog.csdn.net/weixin\\_42608414/article/details/135355723](https://blog.csdn.net/weixin_42608414/article/details/135355723)）：大模型输出的回答是否满足要求（真实性、相关性、表达逻辑）。

逐步实验和写代码的过程是AI项目迭代的过程，最终就可以形成项目经历。

| 模块                                | 检索打分（0-1） | 生成打分（0-1） |
|-----------------------------------|-----------|-----------|
| TFIDF 检索                          | 0.45      |           |
| BERT 检索                           | 0.65      |           |
| 多路合并（TFIDF / BERT）                | 0.70      |           |
| 多路合并（TFIDF / BERT） + Rerank       | 0.75      |           |
| 多路合并（TFIDF / BERT） + Rerank + LLM | 0.75      | 0.56      |
| + 调整 LLM 提示词模板                    |           | 0.59      |
| + 切换 Qwen Max                     |           | 0.64      |
|                                   |           |           |
| 固定长度划分 chunk                      |           |           |
| 固定长度划分 chunk + overlap            |           |           |
|                                   |           |           |

## RAG 最终用户是如何使用的？

## 如何统一处理多文件格式？

## 跨页表格对齐解析如何操作？

## 如何对 CSV（单个表格）/Excel（多个表格）进行问答？

- **读取文件**: 首先，需要使用编程语言（如 Python）中的库（例如 `pandas`）来读取 CSV 或 Excel 文件。这将把表格数据加载到内存中的数据结构（如 DataFrame）。

- **元数据提取**: 提取并存储 CSV/Excel 的元数据（列名及其数据类型）。这些元数据是构建 SQL 语句的关键。
- **解析用户问题**: 接收用户的自然语言问题（例如：“销售额最高的产品是什么？”、“有多少订单来自亚洲？”）。
- **映射到表结构**: 将用户问题中识别出的实体和意图，映射到 CSV/Excel 文件的列名和可执行的操作。
- **生成 SQL/ 类似 SQL 语句**: 根据映射结果，生成针对底层数据（DataFrame 或临时数据库）的查询语句。
- **执行查询**: 在内存中的 DataFrame 或导入的数据库中执行生成的查询语句。

如果excel有100个表，mysql有20个数据库，每个数据库有20个表，怎么办？

- rag 的思路：找到与用户提问最相关的表的信息；

<https://arxiv.org/pdf/2506.10380>

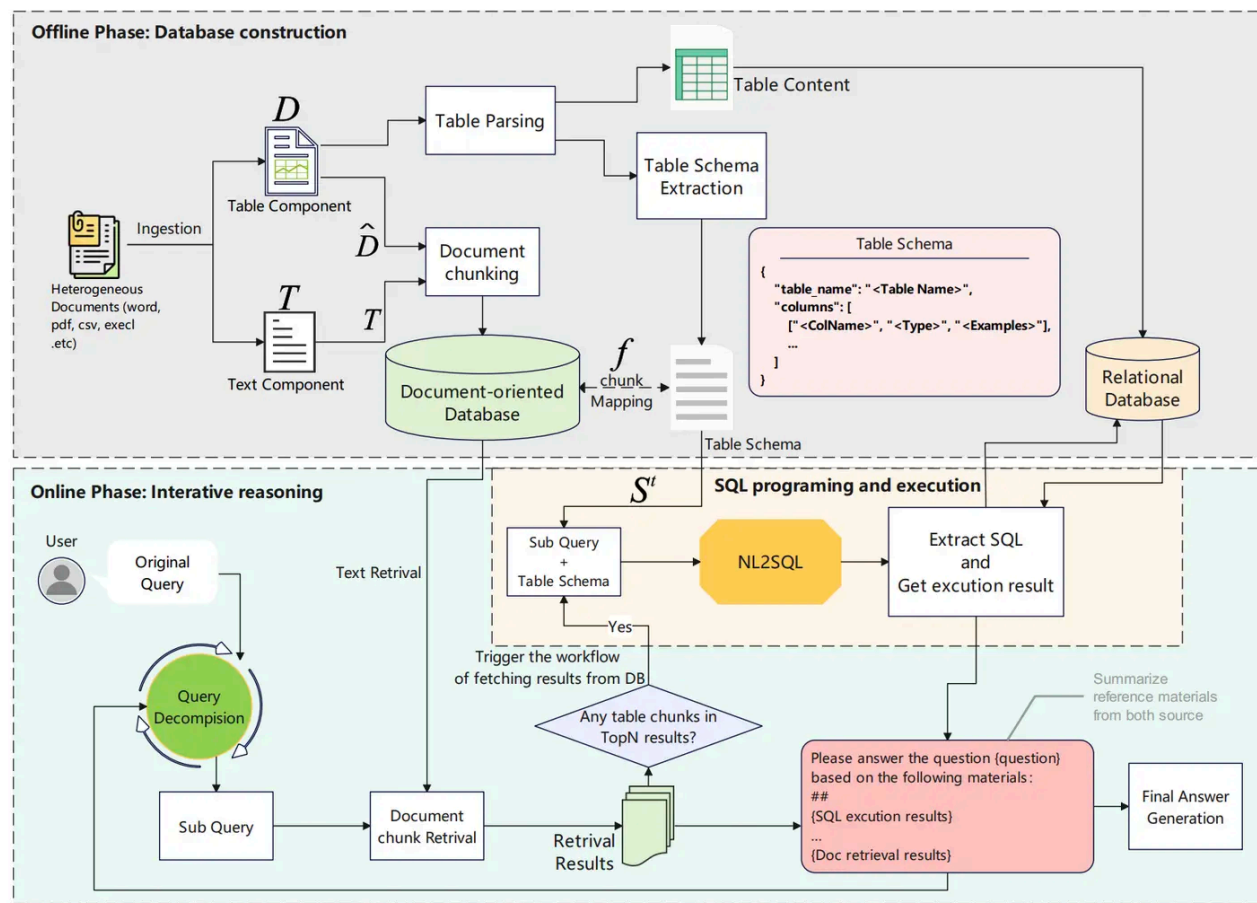


Figure 2: The overall architecture of TableRAG.

## 如何实现图文混排？

首先，需要对原始文档进行**结构化解析**，将文本内容和图像内容有效地分离并存储。

- **文本提取**：从文档中准确提取所有文本。这可能涉及到 OCR（光学字符识别）技术，特别是对于扫描版或非电子文本。
- **图像识别与提取**：识别文档中的所有图像。对于每个图像，建议进行以下处理：
  - **图像上传至对象存储**：将提取出的图像上传至云端对象存储服务（如 AWS S3, 阿里云 OSS 等），并生成可公开访问的 URL。这样可以确保图像的高可用性和加载效率。
  - **图像内容解析与描述生成**：利用图像识别和理解模型（例如多模态 LLM，或专门的图像描述模型）对图像内容进行解析，并生成简洁、准确的文字描述或标签。这些描述将作为后续检索和匹配的重要依据。

在用户提问后，LLM 根据检索到的文本内容，智能地匹配并展示相关的图像。

- 文本检索：利用向量数据库或搜索引擎，根据用户的问题检索出最相关的文本段落。
- 图像关联与筛选：
  - 上下文关联：在检索到相关文本后，检查该文本附近（例如，同一章节、同一页面或前后若干段落）是否存在相关联的图像。这种“上下文”关联是实现图文混排的基础。
  - 图像相似度筛选（可选但推荐）：为了更精确地展示相关图像，可以对之前生成的图像描述进行向量化处理。然后，将用户问题的向量与图像描述的向量进行相似度计算，筛选出与问题最相关的图像。这有助于避免展示不相关的图像，提升用户体验。
  - 多模态理解：更高级的方法是使用多模态 LLM，让模型直接理解问题、文本和图像内容，从而进行更智能的图文匹配。
  - Markdown 格式输出：当确定需要展示图像时，将图像 URL 嵌入到 Markdown 格式的回答中。

## 如何进行图文多模态问答？

方法 1：经过大量图像和文本数据预训练的多模态大语言模型（MLLMs，Qwen-2.5-VL）。这些模型天然地理解视觉信息和文本信息之间的关联。

可以将图像（或多张图像）和文本查询（例如：“这张图中发生了什么？红色按钮的功能是什么？”）直接输入到多模态大模型中。模型会同时处理这两种模态的信息，并生成一个融合了视觉和文本见解的回答。

### Qwen-2.5-VL 代替 Qwen-2.5？

方法 2：描述图像；然后，利用该描述结合纯文本大模型进行问答。

## 何时知道检索的内容足够多？

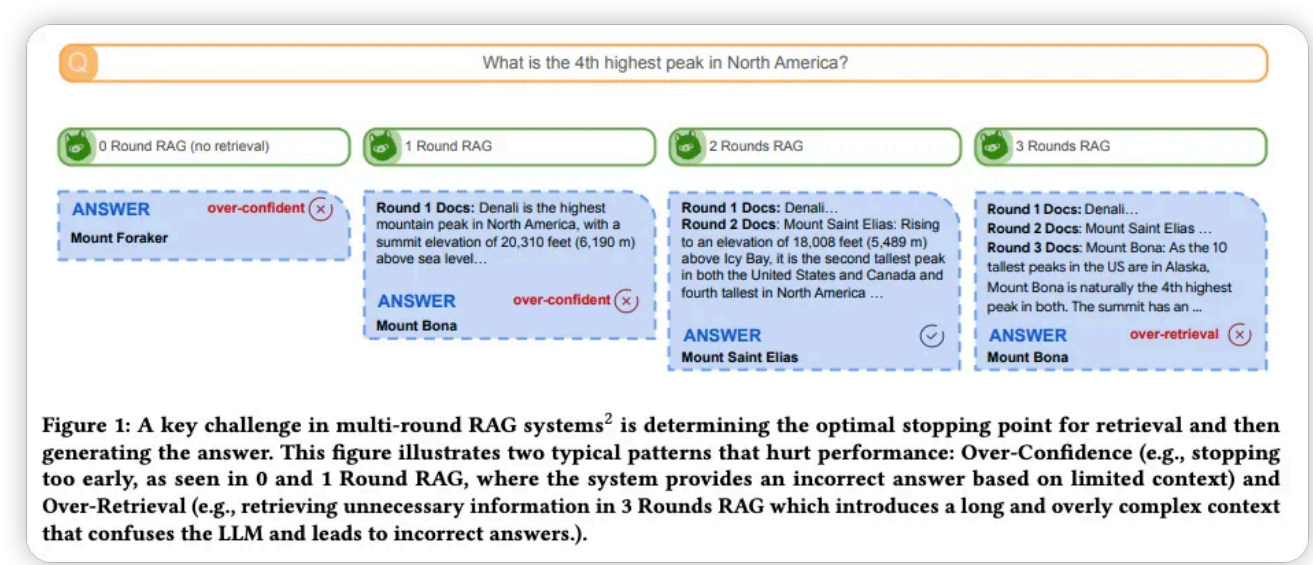
传统的 RAG 系统，也就是单轮 RAG，就像是一个只会“一锤子买卖”的检索员。用户问一个问题，它就用这个问题或者语言模型生成的查询词去知识库中检索一次，然后给出答案。这种系

统对于那些简单直接的问题来说，效果还不错。但一旦遇到需要多步推理、整合多个文档信息或者澄清模糊点的复杂问题，它就无能为力了。

<https://arxiv.org/pdf/2505.02811>

Knowing You Don’ t Know: Learning When to Continue Search in Multi-round RAG through Self-Practicing

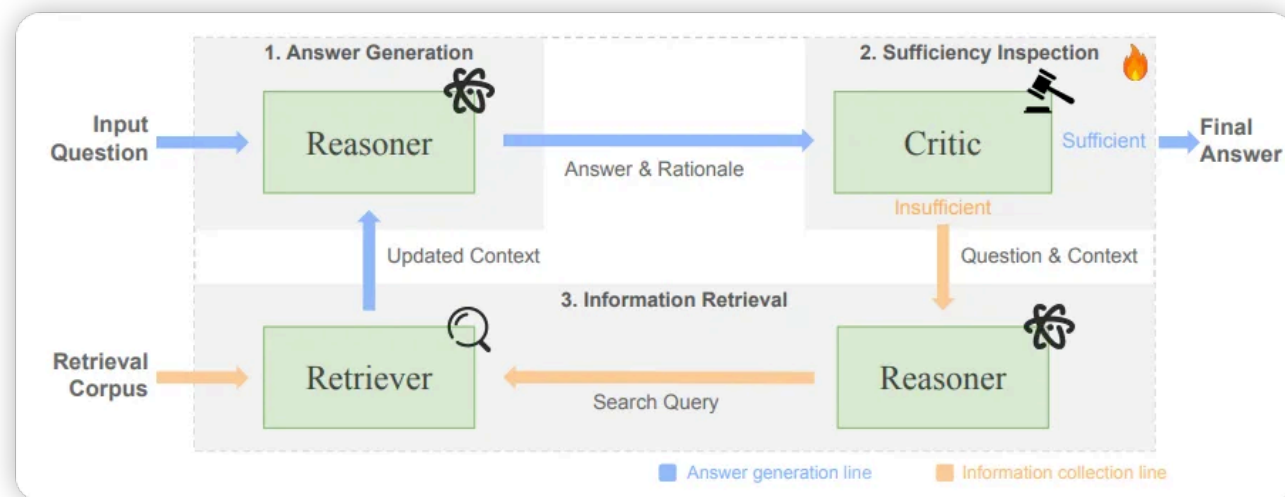
**SIM-RAG（Self-practicing for Inner Monologue-based Retrieval Augmented Generation）** 是一种全新的框架，专门用来提升 AI 的自我意识和多轮检索能力。它的核心思想是让 AI 学会 “知道自己不知道”，就像人类在思考问题时会自我反思一样。



SIM-RAG 的核心思想是模拟人类的“元认知”能力，让 AI 在复杂任务中动态评估自己的知识边界，并根据需要调整检索和推理策略。整个框架由三个主要模块组成：

1. **Reasoner（推理器）**：这是一个语言模型，负责根据当前问题和上下文生成答案和推理过程。
2. **Retriever（检索器）**：这是一个搜索引擎，负责根据 Reasoner 生成的查询词检索外部知识。
3. **Critic（裁判）**：这是一个轻量级的判别模型，负责评估 Reasoner 生成的答案是否足够可靠。

这三个模块协同工作，形成了一个迭代推理框架，模拟人类在面对复杂问题时的思考过程。



## 如何对提问进行针对性回答？

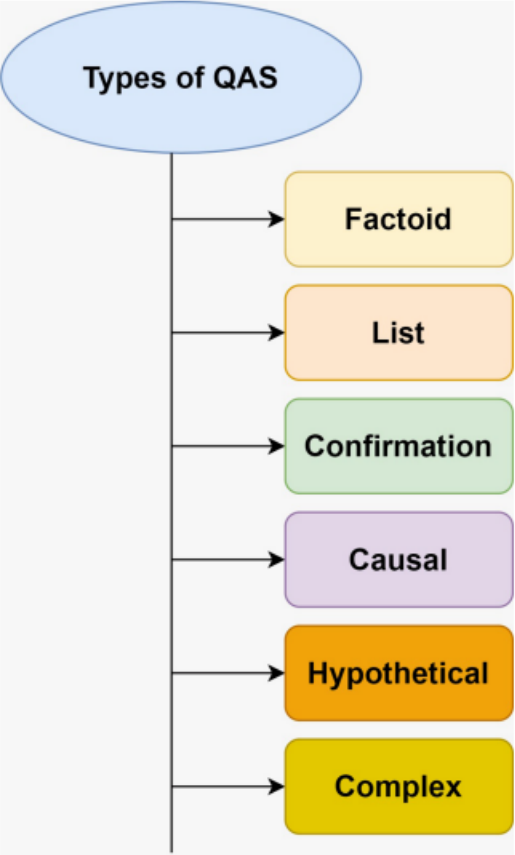
与传统的事实问答（Factoid QA）不同，非事实问答问题没有明确的答案，需要从多个方面进行综合推理，这使得传统的问答方法，包括检索增强生成（Retrieval-Augmented Generation，简称 RAG）方法，都显得有些力不从心。

<https://arxiv.org/pdf/2503.15879>

<https://github.com/TeamNLP/Typed-RAG>

非事实性问题（NFQs）与我们常见的事实性问题（Factoid Questions，简称 FQs）有很大的不同。事实性问题通常有一个明确的答案，比如“法国的首都是哪里？”答案就是“巴黎”。而 NFQs 的答案往往不是唯一的，比如“人工智能对社会的利弊”，这个问题可能需要从技术、经济、伦理等多个角度来回答。这种复杂性使得传统的问答系统很难满足用户的需求。

现有的非事实性问答方法（NFQA）虽然在某些方面有所改进，但仍然存在一些问题。比如，它们很难处理多样化的 NFQs，也没有充分利用大型语言模型（LLMs）和检索增强生成（RAG）框架。标准的 RAG 方法虽然能够增强回答的上下文相关性，但无法应对 NFQs 的多样性，导致生成的回答过于单一，缺乏多方面的深度。



- 事实性问题（FQs）只需要一个简单的、直接的答案，比如“法国的首都是哪里？”答案就是“巴黎”。
- 非事实性问题（NFQs）。需要更复杂、更主观或更全面的回答。

此外还有更加细致的划分：

- 指导型（Instruction）
- 原因型（Reason）
- 基于证据型（Evidence-based）
- 比较型（Comparison）
- 经验型（Experience）
- 辩论型（Debate）

Typed-RAG 的核心在于将非事实性问题（NFQs）分类，并针对每种类型的问题采用不同的处理策略。NFQs 的类型多样，意图不同，回答的角度也不同。**有些问题需要对比，有些需要经验分享，有些则需要辩论。Typed-RAG 通过多方面分解，将复杂的 NFQs 分解为多个单方面子问题，分别处理后再进行聚合，生成全面且精准的回答。**

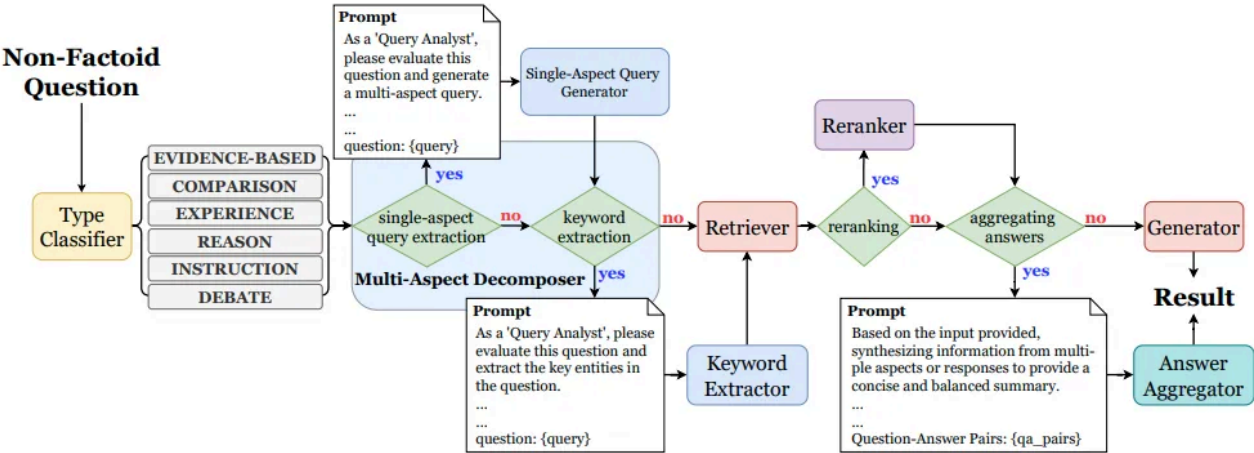
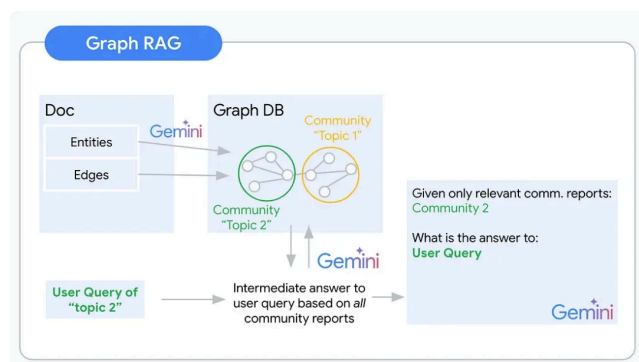


Figure 1: An overview of Typed-RAG. Non-factoid questions are classified by the type classifier and processed based on their type. Prompts for the multi-aspect decomposer and answer aggregator handle the unique requirements of each type. Details of the prompt can be found in Appendix A.4.

| 问题类型                             | 问题特点                                                                                   | 回答方式                                                                                                                                                                                                                                                              |
|----------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 基于证据型（Evidence-based）问题          | 这类问题通常要求对特定概念、对象或事件的特征或定义进行解释，需要基于事实的回答。例如，“人工智能的定义是什么？”这类问题通常只需要从一个角度回答，因此不需要进行多方面分解。 | 对于基于证据型问题，Typed-RAG 采用直接的 RAG 方法。检索器（Retriever）以问题为查询，检索相关的段落；生成器（Generator）则基于这些段落生成清晰、简洁且准确的回答。                                                                                                                                                                 |
| 比较型（Comparison）问题                | 这类问题用于比较关键词之间的差异、相似性或优劣。例如，“人工智能和机器学习有什么区别？”这类问题需要从多个角度进行回答，因此需要进行多方面分解。               | <ol style="list-style-type: none"><li><b>关键词提取：</b>首先，关键词提取器（Keyword Extractor）识别比较的目的（compare_type）和目标（keywords_list）。</li><li><b>检索与去重：</b>检索器根据每个关键词检索相关段落，并去除重复内容。</li><li><b>重新排序：</b>根据相关性对剩余段落进行重新排序。</li><li><b>生成回答：</b>生成器结合这些信息，生成符合比较目的的回答。</li></ol> |
| 经验型（Experience）问题                | 这类问题通常寻求建议或推荐，基于个人经验进行解释。例如，“如何提高编程能力？”这类问题需要从多个角度提供经验分享。                              | <ol style="list-style-type: none"><li><b>检索相关段落：</b>检索器检索与问题相关的段落。</li><li><b>基于相似性的重新排序：</b>关键词提取器提取关键词后，根据相似性对段落进行重新排序。</li><li><b>生成回答：</b>生成器根据检索到的段落生成符合问题意图的回答。</li></ol>                                                                                   |
| 原因型（Reason） / 指导型（Instruction）问题 | 原因型问题旨在探索特定现象的原因，指导型问题则关注程序或方法的理解。例如，“为什么人工智能发展如此迅速？”或“如何学习人工智能？”这类问题需要清晰、全面且结构化的回答。   | <ol style="list-style-type: none"><li><b>多方面分解：</b>将问题分解为多个单方面子问题。</li><li><b>分别处理：</b>每个子问题分别由检索器检索相关段落，生成器生成回答。</li><li><b>聚合答案：</b>答案聚合器（Answer Aggregator）将这些单独的回答合并为一个简洁、准确的最终回答。</li></ol>                                                                  |
| 辩论型（Debate）问题                    | 这类问题是假设性问题，旨在探索多种视角，包括对立的观点。例如，“人工智能是否对社会有益？”这类问题需要生成一个平衡的回答，反映多种视角。                   | <ol style="list-style-type: none"><li><b>多方面分解：</b>将问题分解为多个单方面子问题。</li><li><b>分别处理：</b>每个子问题分别由检索器检索相关段落，生成器生成回答。</li><li><b>聚合答案：</b>辩论调解器（Debate Mediator）将主题和各种观点结合起来，生成一个平衡的最终回答。</li></ol>                                                                   |

需要尝试图谱 RAG 吗？



图谱 RAG 通过引入图结构化数据来增强基础 RAG 的能力：

- 图结构化索引：利用图数据库存储和检索信息，通过图结构捕捉数据之间的复杂关系。
- 双级检索策略：结合低层次的具体信息检索和高层次的广泛话题检索，提高检索的全面性和准确性。
- 动态更新：支持增量更新算法，能够快速适应新数据

## A Pilot Empirical Study on When and How to Use Knowledge Graphs as Retrieval Augmented Generation

<https://arxiv.org/pdf/2502.20854>

要回答“何时使用 KG-RAG”的问题，首先需要考虑任务场景。论文从两个角度对任务场景进行了分类：任务领域和任务难度。

- **开放域问答 (Open-domain QA)**：这类任务需要通用的世界知识，例如“地球的自转方向是什么？”。
- **特定领域问答 (Domain-specific QA)**：这类任务需要专业知识，例如医学咨询“我皮肤上有奇怪的疙瘩，会不会是病毒疣？”。
- **专业考试 (Domain-specific Exam)**：这类任务是针对特定领域的资格考试，例如医学专业考试。

任务难度的划分目前还没有统一的标准。论文采用了两级分类：

- **L1 难度**：只需要基于明确事实的简单答案（单跳问题），例如“北京是中国的首都吗？”。
- **L2 及以上难度**：需要推理和整合多条信息（多跳问题），例如“为什么说维生素 C 可以增强免疫力？”

在特定领域任务中，KG-RAG 能够显著增强开源 LLM（如 Llama2-7B）的表现，使其在某些场景下甚至可以与商业 LLM 相媲美。当在开放域问答中，KG-RAG 的提升效果相对有限。这可能是由于开放域任务需要更广泛的知识覆盖，而当前的 KG-RAG 方法在知识广度上仍有不足。

## 如何训练一个重排序模型？（项目主流程完成之后，优化性能）

训练一个强大的重排序模型就像搭建一座桥梁，需要多个关键组件的协同合作。

- 如何准备数据集？
- 如何选择损失函数？
- 为什么训练参数重要？
- 为什么训练器重要？

<https://huggingface.co/blog/train-reranker>

如 Hugging Face Datasets Hub 是一个强大的数据集资源库，提供了大量的公开数据集，这些数据集可以直接用于训练和评估重排序模型。许多数据集已经被标注为与 Sentence Transformers 兼容，这意味着它们可以直接用于训练重排序模型。

Python

```
1 from datasets import load_dataset
2
3 train_dataset = load_dataset("sentence-transformers/natural-questions", split="train")
4
5 print(train_dataset)
6 """
7 Dataset({
8     features: ['query', 'answer'],
9     num_rows: 100231
10 })
11 """
```

在训练重排序模型时，负样本的质量往往决定了模型的性能。负样本是指那些与查询不相关的文本片段，模型需要学会将这些片段的得分降低。负样本可以分为两类：软负样本和硬负样本。

- **软负样本 (Soft Negatives)**：与查询完全无关的文本片段，也称为“容易的负样本”。例如，查询是“苹果公司在哪里成立的？”软负样本可能是“阿肯色州的 Cache River Bridge 是一座跨越 Cache River 的桥梁。”
- **硬负样本 (Hard Negatives)**：看起来可能与查询相关，但实际上并不相关的文本片段。例如，查询是“苹果公司在哪里成立的？”硬负样本可能是“富士苹果是一种在 20 世纪 30 年代末开发的苹果品种，于 1962 年推向市场。”

Python

```
1 from datasets import load_dataset
2 from sentence_transformers import SentenceTransformer
3 from sentence_transformers.util import mine_hard_negatives
4
5 # Load the GooAQ dataset: https://huggingface.co/datasets/sentence-transformers/gooaq
6 train_dataset = load_dataset("sentence-transformers/gooaq", split="train").select(range(100_000))
7 print(train_dataset)
8
9 # Mine hard negatives using a very efficient embedding model
10 embedding_model = SentenceTransformer("sentence-transformers/static-retrieval-mrl-en-v1", device="cpu")
11 hard_train_dataset = mine_hard_negatives(
12     train_dataset,
13     embedding_model,
14     num_negatives=5, # How many negatives per question-answer pair
15     range_min=10, # Skip the x most similar samples
16     range_max=100, # Consider only the x most similar samples
17     max_score=0.8, # Only consider samples with a similarity score of
    at most x
18     margin=0.1, # Similarity between query and negative samples should be x lower than query-positive similarity
19     sampling_strategy="top", # Randomly sample negatives from the range
20     batch_size=4096, # Use a batch size of 4096 for the embedding model
21     output_format="labeled-pair", # The output format is (query, passage, label), as required by BinaryCrossEntropyLoss
22     use_faiss=True, # Using FAISS is recommended to keep memory usage low (pip install faiss-gpu or pip install faiss-cpu)
23 )
24 print(hard_train_dataset)
25 print(hard_train_dataset[1])
```

损失函数是训练过程中的“指南针”，它告诉模型哪些方向是正确的，哪些是错误的。具体来说，损失函数的作用包括：

- 1. **衡量模型性能：**损失函数通过计算模型的输出和真实标签之间的差异，来衡量模型的性能。差异越小，说明模型的性能越好。
- 2. **指导优化过程：**损失函数的值会通过优化算法（如梯度下降）来调整模型的参数，使损失值最小化。不同的损失函数可能导致模型学习到不同的特征。

多数据集训练是提升通用模型性能的有效方法。通过在多个数据集上同时训练，模型可以学习到更广泛的数据特征，从而提高泛化能力。 `CrossEncoderTrainer` 支持多数据集训练，并且允许对每个数据集使用不同的损失函数。

- 3. **准备数据集：**使用一个字典来存储多个数据集，字典的键是数据集的名称，值是 `datasets.Dataset` 实例。
- 4. **选择损失函数：**如果需要对每个数据集使用不同的损失函数，可以准备一个字典，将数据集名称映射到对应的损失函数。
- 5. **设置采样策略：**通过 `CrossEncoderTrainingArguments` 的 `multi_dataset_batch_sampler` 参数选择采样策略。

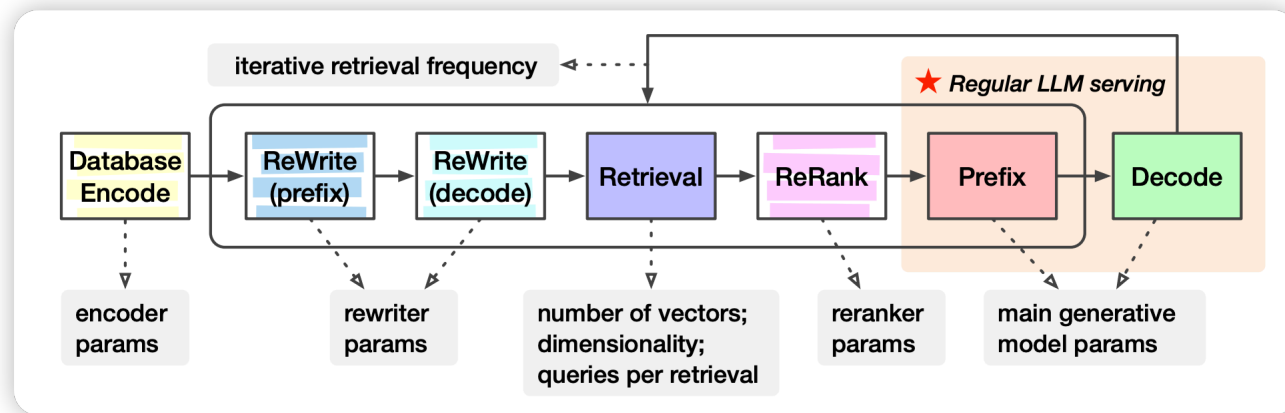
## 如何优化大模型应用（RAG 系统）性能？

RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation  
Serving

<https://arxiv.org/pdf/2503.14649>

文章对四种具有代表性的 RAG 工作负载进行了详细的分析，揭示了它们在性能上的巨大差异。这些工作负载包括：

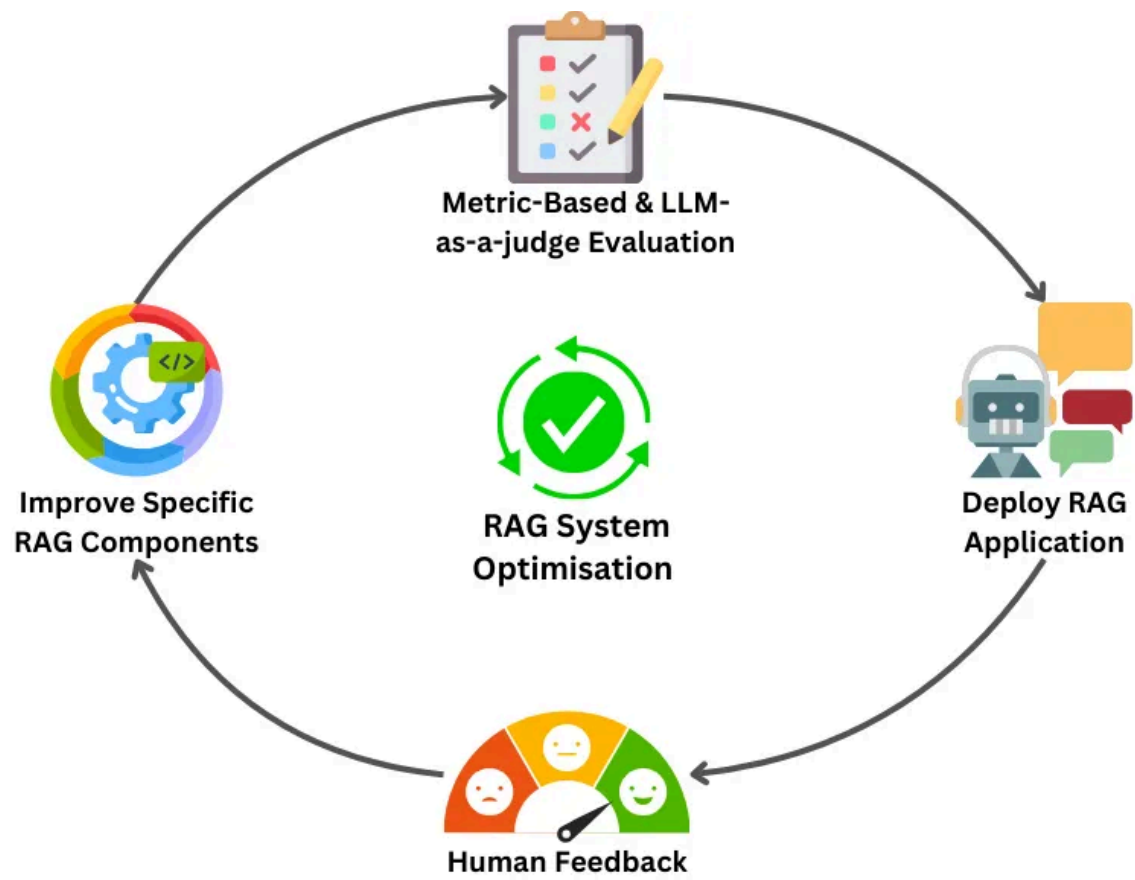
- 1. **具有超大规模检索的 RAG：**在这种情况下，检索过程可能占据超过 80% 的总延迟，成为明显的瓶颈。
- 2. **用于长上下文序列处理的 RAG：**与超大规模检索相反，在长上下文场景中，检索过程的延迟占比不到 1%，而其他部分则可能成为性能瓶颈。
- 3. **具有迭代检索的 RAG：**在解码过程中进行迭代检索可能会阻塞整个流水线，因为解码过程需要等待检索结果。
- 4. **带有查询重写器和检索结果重排器模型的 RAG：**即使是流水线中较小的模型，如数据库编码器，也可能因为需要处理大量标记而成为瓶颈。



RAGSchema 定义了 RAG 流水线的执行流程和组件配置。它包括以下几个关键部分：

- **文档编码器**：用于将数据库文档和查询转换为向量表示的模型大小。
- **向量维度**：每个数据库向量的维度。
- **数据库向量数量**：取决于语料库大小和片段长度。
- **检索频率**：是否允许在解码过程中进行迭代检索，以及每次序列的检索次数。
- **每次检索的查询向量数量**：每次检索中使用的查询向量数量。
- **查询重写器**：用于重写用户查询的模型大小。
- **结果重排器**：用于对检索结果进行评分的模型大小。
- **生成型 LLM**：用于生成回答的主 LLM 的模型大小。
- **LLM 使用次数**：在回答问题的过程中需要调用 LLM 的次数。

如何优化 RAG 回答的效果（体验、回答的效果）？



---

## 附录

### 专业名词介绍

| 专业名词              | 描述                                                                                                 |
|-------------------|----------------------------------------------------------------------------------------------------|
| 大型语言模型            | 在自然语言处理领域展示出强大生成能力的模型，如 GPT 系列。但其修改、微调或重新训练困难，成本高。                                                 |
| Prompt            | "Prompt"（提示）是指一种引导大型语言模型（LLM）生成特定文本的方法。或可以理解为输给大模型的输入文本。                                           |
| 幻觉（Hallucination） | 在自然语言处理领域被定义为生成的内容与提供的源内容无关或不忠实，一种虚假的感知。                                                           |
| 知识库问答（KBQA）       | 早期的对话系统方法，利用结构化的知识库进行自然语言问题的回答。知识库以三元组形式表示 < 主题，关系，对象 >，存储在图数据库中。                                  |
| RAG               | RAG 是检索增强生成（Retrieval-augmented Generation）的缩写，是一种结合了大型语言模型的生成能力和检索系统的精确性的技术，用于提高生成内容的准确性、相关性和时效性。 |
| 倒排索引              | 倒排索引（Inverted Index）是一种数据结构，用于加速文本检索过程。它将文档中的词汇映射到出现该词汇的文档列表，从而实现根据词汇快速检索相关文档的目的。                  |
| 文本嵌入              | 文本嵌入是将文本信息映射到高维向量空间的过程，使得具有语义相似性的文本在向量空间中距离较近。                                                     |
| 文本相似度             | 文本相似度是衡量两段文本之间语义接近程度的度量。通过计算文本在嵌入空间中的相似性，可以评估它们在语义上的相似程度。                                          |
| 排序与重排序            | 在信息检索中，排序指的是将检索到的文档按照其与查询的相关性进行排序。重排序则是在排序后的结果基础上再次调整文档的顺序，以进一步提高与用户查询的匹配度。                        |

## RAG 框架

## 文档解析工具

步骤 1、第三方工具部署为 web 服务

pdf 解析后：文本、图片、表格 -》 文本存储 -》 入库

步骤 2、定义为 dify 第三方工具

步骤 3、上传逻辑（选择上传到什么知识库？）

## PaddleOCR 开源

<https://github.com/PaddlePaddle/PaddleOCR>

## **MinerU 开源**

<https://github.com/opendatalab/MinerU>

## **MathPix 商业**

## **Qwen-VL 开源**

## **Unstructured 格式比较多**

<https://docs.unstructured.io/welcome>